



東華理工大學

EAST CHINA UNIVERSITY OF TECHNOLOGY

毕 业 设 计

题 目 基于 Node.js 和 react 的社区助老服务对接平台的设计与实现

英文题目 Design and Implementation of a
Community-Based Elderly Support Service
Integration Platform Based on Node.js and React

学生姓名: 汪序磊 申请学位门类: 工学

学 号: 2022213086

专 业: 软件工程

学 院: 软件学院

指导教师: 余小军 职称: 讲师

二〇二六年六月三十日



東華理工大學

EAST CHINA UNIVERSITY OF TECHNOLOGY

Graduation Design

Design and Implementation of a Community-Based Elderly Support
Service Integration Platform Based on Node.js and React

StudentName: Wang Xulei

DegreeCategory: Engineering

Student ID: 2022213086

Major: Software Engineering

Major: School of Software Engineering

InstructorName: Yu Xiaojun

ProfessionalTitles: Lecturer

Date: 2026-06-30

作者声明

本人以信誉郑重声明：所呈交的学位毕业设计（论文），是本人在指导教师指导下由本人独立撰写完成的，没有剽窃、抄袭、造假等违反道德、学术规范和其他侵权行为。文中引用他人的文献、数据、图件、资料均已明确标注出，不包含他人成果及为获得东华理工大学或其他教育机构的学位或证书而使用过的材料。对本设计（论文）的研究作出重要贡献的个人和集体，均已在文中以明确方式标明。本毕业设计（论文）引起的法律结果完全由本人承担。

本毕业设计（论文）成果归东华理工大学所有。

特此声明。

毕业设计（论文）作者（签字）：

签字日期： 年 月 日

本人声明：该学位论文是本人指导学生完成的科研成果，已经审阅过论文的全部内容，并能够保证题目、关键词、摘要部分中英文内容的一致性和准确性。

学位论文指导教师签名：

 年 月 日

摘 要

伴随着我国人口老龄化的加剧,社区养老服务的需求也由原来的单一照护模式向数字化、精细化、协同化服务模式转变。老年人在代办代购、陪同就医、生活照料、精神陪伴等各方面的现实需求不断增长,社区养老资源供给分散、信息沟通不畅、服务响应效率不高等问题仍然比较突出,因此建立智慧助老服务对接平台有很强的现实必要性。

本文以社区助老服务对接场景为研究背景,从老年人、家属、志愿者、社区管理员这四个主要用户出发,设计并实现了一个基于 Node.js、React 的社区服务对接平台。系统采用前后端分离架构,前端以 React、TypeScript 和 Ant Design 实现多角色业务界面,后端基于 Node.js 与 Express 搭建 RESTful 服务,并结合 MySQL、Redis、Socket.io、高德地图 API 及 ECharts 等技术完成数据存储、实时通信、地图展示与统计分析。

本文主要工作有以下几方面内容,对社区助老服务的业务场景以及用户需求进行分析,完成平台可行性分析、功能需求分析和业务流程设计,以需求发布、任务接单、订单跟踪、志愿积分、后台审核和数据看板等模块为基础进行系统设计与实现,最后通过测试验证平台的主要功能可用性和业务完整性,给社区养老服务的信息化建设提供可以落地的技术参考。

关键词: 智慧助老; 社区养老; Node.js; React; 服务对接平台

ABSTRACT

With the increase in the number of older people in China, community elderly-care services are now developing a trend of digitalisation, refinement and cooperation. With the aging population, more older people and their families need daily assistance, medical care, home-based care and emotional support, but the resources from community services are still scattered and poorly organised. Therefore, a smart elderly care service matching platform will have good application value.

The scenario of community "smart elderly assistance" service integration is the research background of this paper. Based on the four main groups of users, that is to say older people, family members, volunteers and community administrators, a community service integration platform has been designed and built with Node.js and React. The system is a separation of the front-end and back-end, and React, TypeScript, and Ant Design are employed for the front-end to build various business interfaces; Node.js and Express have been chosen for the back-end to provide RESTful services, and MySQL, Redis, Socket.io, AutoNavi Map API, ECharts, etc., are utilized for data storage, real-time communication, map visualisation, statistics, and so on.

The main purpose of this paper is to study the operating environment and user needs of community-based smart elderly assistance services, carry out a feasibility study, conduct functional requirement analysis and business process design for the platform, develop and implement a system with modules such as requirement release, task acceptance, order tracking, volunteer point management, backend review and data dashboard, etc., and finally test the usability and operational completeness of the core functions of the platform to provide practical technical support for the digitalisation of community elderly care services.

Keywords: Smart Elderly Care Service Matching Platform; Community Elderly Care; Node.js; React; Service Matching

目 录

第 1 章 绪论1

1.1 开发背景、目的和意义1

1.2 国内外研究现状1

1.2.1 国外研究现状2

1.2.2 国内研究现状2

1.3 主要研究内容3

第 2 章 相关技术介绍5

2.1 REACT 框架5

2.2 NODE.JS 与 EXPRESS5

2.3 REDUX TOOLKIT5

2.4 ANT DESIGN6

2.5 SOCKET.IO6

2.6 MYSQL 与 REDIS6

2.7 高德地图 API 与 ECHARTS7

第 3 章 系统需求分析8

3.1 可行性分析8

3.1.1 技术可行性8

3.1.2 操作可行性8

3.1.3 经济可行性9

3.2 功能需求分析9

3.3 业务流程分析11

3.4 数据流程分析12

第 4 章 系统详细设计14

4.1 系统架构14

4.2 系统功能模块设计15

4.3 系统用户设计16

4.4 数据库设计17

4.4.1 概念模型设计	17
4.4.2 数据库表设计	18
第 5 章 系统实现	22
5.1 登录界面	22
5.2 老年人/家属端首页	22
5.3 需求发布与订单跟踪	24
5.4 志愿者任务大厅	29
5.5 志愿服务执行与积分	31
5.6 管理员数据看板	35
5.7 志愿者审核管理	38
5.8 需求审核与工单调度	40
5.9 统计分析	42
5.10 个人信息与老人档案	45
第 6 章 系统测试	51
6.1 测试的目的	51
6.2 测试方法	51
6.3 测试用例	52
6.4 测试结论	55
结 论	57
致 谢	58
参考文献	59

第1章 绪论

1.1 开发背景、目的和意义

近年来,我国人口老龄化程度持续加深,老年人口规模不断扩大,养老服务需求也由基本生活照料逐步延伸到就医陪护、精神慰藉、紧急求助、日常代办、康复支持等多个方面。与此相对应,社区作为养老服务落地最直接、最基础的场景,正在承担越来越重要的功能。但从现实情况看,当前不少社区养老服务仍然存在信息分散、需求上报不及时、任务分配效率不高、服务过程难跟踪、服务结果难沉淀等问题,传统依赖电话、微信群、纸质登记和熟人对接的方式,已经难以满足高频化、碎片化、即时化的养老服务需求。因此,借助数字化手段建设面向社区的助老服务对接平台,已成为提升基层养老服务能力的重要方向。

从政策背景来看,国家近年来持续把积极应对人口老龄化上升为重要战略任务,明确提出实施积极应对人口老龄化国家战略,完善养老服务体系,推动养老事业和养老产业协同发展。《“十四五”国家老龄事业发展和养老服务体系规划》强调,要加快健全居家社区机构相协调、医养康养相结合的养老服务体系;《关于发展银发经济增进老年人福祉的意见》进一步提出,要围绕老年群体需求培育高质量服务模式,推进养老服务提质扩容;《中共中央国务院关于深化养老服务改革发展的意见》也对完善养老服务网络、增强服务供给能力、提升服务质量提出了更高要求。可以看出,国家政策导向已经十分明确,即通过制度完善、数字赋能和资源整合,不断提高养老服务的可及性、便利性和精准性。本项目围绕社区助老服务展开设计与实现,符合国家关于积极应对人口老龄化、发展银发经济、推进智慧养老和完善基层治理的政策方向。

从项目作用来看,社区助老服务对接平台的建设,能够把老年人、家属、志愿者、社区工作人员等多方主体纳入统一平台之中,实现需求发布、任务匹配、过程跟踪、结果反馈和数据统计的闭环管理。一方面,该平台有助于提升社区养老服务的响应速度和资源配置效率,缓解人口老龄化背景下社区服务压力不断上升的问题;另一方面,也有助于推动养老服务由经验驱动向数据辅助决策转变,增强服务透明度和持续改进能力。因而,本项目不仅具有较强的现实应用价值,也体现了对国家发展方针和养老服务改革方向的积极响应,对于提升社区养老服务质量、促进基层治理现代化具有较为明确的意义。

1.2 国内外研究现状

近些年来,国内外有关智慧助老及社区养老服务平台方面的研究持续深化,

研究重点逐渐从单一的信息展示和分散的人工协调,转向围绕真实养老场景展开的平台建设与服务协同。整体来看,现有研究越来越关注服务资源整合、需求响应效率、过程跟踪能力以及多主体之间的协同机制,平台建设思路也由单点功能实现逐步走向系统化、场景化和闭环化。

从研究内容来看,相关成果大致可以归纳为三个层面:一是围绕社区服务流程展开的平台建设研究,重点关注服务入口、需求发布、任务分派和资源衔接;二是围绕系统实现展开的技术研究,重点关注平台架构、数据组织、移动端承载和智能感知能力;三是围绕老年用户展开的适老化研究,重点关注界面理解、操作便利性、使用支持和推广可行性。基于此可以认为,智慧助老平台并不是单一的软件工具,而是技术、服务与治理相互结合的综合载体。

1.2.1 国内研究现状

就国内研究现状而言,相关研究首先关注智慧助老平台建设的总体方向与场景定位。朱培垚、乔雯、张佳萌指出,智慧养老产业发展需要兼顾资源整合与可信流转机制[1]。莫文卓等在“乐龄伴行”智慧助老 APP 的开发实践中发现,老年用户对界面简洁性、操作反馈和陪伴感有较强需求[2]。凌超等对智慧助老平台的设计与实现研究也表明,社区助老系统应具备需求汇聚、任务分发和过程跟踪能力[3]。沈进兵则从社区教育与社会行动协同的角度说明,智慧助老平台建设不能只理解为技术部署问题[4]。

在服务入口与具体应用实现方面,国内研究已经形成了较多可借鉴成果。彭敏学等提出的社区智慧助老信息服务系统说明,轻量化入口有助于提升社区服务的可获得性[5]。钱荣华指出,线上平台能够扩大助老服务的触达范围[6]。傅俊哲、李俊杰认为,小程序形态在保留清晰入口和简化操作流程方面具有现实优势[7]。这些研究说明,国内平台建设已经开始重视服务流程与使用路径的整体优化。

从老年用户体验和适老化支持来看,耿梦瑶、杨宁强调,助老产品应尽量降低理解负担并提升交互友好度[8]。赵敏认为,提升老年人智能手机使用能力是智慧助老平台落地的重要前提[9]。王皖琳等对老年群体“数字鸿沟”的分析也表明,平台建设还需要兼顾适老化改造与使用支持[10]。在平台架构与系统实现层面,赵莹德认为,以物联网为基础的智慧社区服务平台可以提高社区资源感知和服务响应速度[11]。张致瑜等指出,智慧社区服务体系建设应实现平台化整合和多主体协同[12]。

进一步看,樊斐从微服务架构角度论证了社区服务平台在扩展性和业务闭环方面的可行性[13]。王晨的研究也说明,多角色业务的统一入口和移动端承载方式具有较强的现实可行性[14]。靳冰、平雷关于 AI 与 IoT 融合的讨论表明,感知技术与平台管理能力结合后有助于提升智能协同水平[15]。张凯关于资源组织

与连接的分析进一步说明,社区养老服务需要依托平台实现高效衔接[16]。温璐亚和赵袁杰关于责任追溯与数据留痕的建议,为平台可信运行提供了补充[17]。周红婕关于平台运营治理的研究表明,治理能力会影响服务触达效率和持续运行效果[18]。高天关于整体架构的研究,也进一步丰富了智慧助老平台的研究基础[19]。总体来看,国内研究已经在社区服务落地、平台架构设计、过程治理和适老化改造等方面形成了较为完整的认识。

1.2.2 国外研究现状

就国外研究现状而言,相关成果更加重视智慧养老平台在政策支持、连续照护、需求动态响应和智能感知等方面的系统性建设。Shi A、Zhao Y、Qi Q 等认为,智慧养老服务平台需要在政策支持下具备对老年群体差异化需求进行动态响应和资源协调的能力[20]。Tan X、Su X 关于连续护理服务平台的研究表明,平台还应为老年人提供院后家庭照护、健康监测和连续性保障机制[21]。Liu Q、Jia L、Yan Q 则指出,智能感知技术能够提升养老平台在信息采集、状态识别和服务触发方面的准确性[22]。

从国外研究思路来看,平台建设并不是把若干服务功能简单叠加起来,而是强调需求识别、持续服务和治理机制之间的联动关系。换句话说,平台既要能在前端承接服务申请,也要能在中后端完成状态跟踪、资源协调和服务反馈,从而形成较为完整的运行闭环。

此外,国外研究通常更加注重跨主体协同和长期服务连续性问题,例如如何让家庭、社区、医疗照护以及平台运营方围绕同一条服务链条协同工作,如何通过数据采集与状态识别提高响应及时性,以及如何把持续照护能力嵌入平台机制之中。这些研究说明,智慧养老平台的价值不仅体现在“能不能提供服务”,也体现在“能不能稳定地持续提供服务”。

但是,将国外研究成果转化到国内社区助老服务场景时,还需要考虑本土社区治理模式、基层服务资源分布和老年用户数字能力水平等现实条件。因此,国外研究对于本文的启示主要体现在平台闭环设计、状态追踪、资源协同和持续服务保障等方面,而不是对既有实现路径的机械照搬。

1.3 主要研究内容

本文以基于 Node.js 和 React 的社区助老服务对接平台为研究对象,从社区养老服务中需求发布、资源匹配、服务执行、过程反馈、运营管理等主要业务入手进行分析、设计、实现。本文在了解课题研究背景的基础上,从课题研究的目的是和意义入手,同时又分析了国内外的研究状况,从而得出本课题的研究目标和技术路线。

根据开题报告、任务书和项目已有的实现内容,从老年人/家属端、志愿者

端、管理员端三个方面对系统功能需求、业务流程、数据流程进行分析，从而完成系统的架构、功能模块、用户设计、数据库结构的设计。就社区助老场景下信息流转链条长、角色参与主体多的特性而言，本文主要对平台的流程闭环、状态同步以及角色协同等进行研究。

本文以登录注册、需求发布、任务大厅、订单跟踪、双端实时聊天、积分奖励机制、后台审核调度、数据看板、老人档案管理等主要模块为依托，对系统的实现进行了详细的说明，同时从页面展示、功能流程、关键业务逻辑三个方面对系统运行的过程进行了分析。

最后本文利用系统测试的方法对平台的可用性、业务完整性和基本稳定性进行系统的测试，在此基础上对社区助老服务对接平台未来功能扩展、适老化改造和数据治理改善等提出一些思考。

第 2 章 相关技术介绍

2.1 REACT 框架

React 属于组件化的前端框架，可以创建出交互频繁且角色复杂的多页面应用系统。本项目中 React 主要用来搭建老年人/家属端、志愿者端、管理员端页面，用组件拆分来实现登录、注册、需求发布、订单列表、聊天面板、统计卡片等界面的复用。

就本项目实现要求而言，React 的优势不单表现在组件的复用性上，更主要地表现在它对于复杂的界面状态和交互流程有着很好的支持能力上。社区智慧助老服务对接平台包含老年人或者家属端、志愿者端以及管理员端这三种角色，各个角色所对应的页面结构、功能入口以及数据展示方式均有所不同。使用 React 进行开发，可以把登录注册、需求表单、任务列表、聊天窗口、统计面板等拆分成独立的组件，在保证界面一致性的基础上提高开发效率，也便于以后根据业务的变化进行局部的迭代以及模块的扩展。

2.2 NODE.JS 与 EXPRESS

Node.js 采用事件驱动、非阻塞 I/O 模式，可以很好地处理订单状态变更、消息通知以及接口并发请求这些业务情况。Express 属于轻量级 Web 开发框架，担负起路由分发、请求处理以及接口组织等工作任务，给平台后端的认证授权、需求管控、订单流转和后台审核赋予稳固支撑。

本系统后端采用 Node.js 和 Express 的组合方式可以较好地适应社区助老服务平台接口多、状态流转快、实时交互需求高等特点。Node.js 对异步请求以及高并发访问的处理比较强，适合用在登录认证、需求提交、订单状态修改、消息通知这些地方。Express 用中间件、路由组织能力、灵活的接口封装方式来构建清晰的服务层结构，把认证授权、参数校验、业务处理、错误响应等按照统一规范组织起来，提高后端代码的可维护性。

2.3 REDUX TOOLKIT

Redux Toolkit 用于管理前端全局状态，特别是用来维护用户的登录状态、角色身份以及跨页面共享的数据。本项目中主要用作统一保存当前会话信息以及角色上下文，保证用户在页面跳转或者刷新之后仍然保持一致的业务视图。

本项目属于多角色单页应用，前端状态管理直接影响到页面之间数据的一致性以及交互的稳定性。Redux Toolkit 用简化的状态切片定义、异步逻辑处理、全局状态读取的方式，使用户登录信息、角色权限、聊天会话上下文、部分共享业

务数据在各个页面之间可以稳定地流转。相比零散地使用局部状态管理，集中式的状态组织方式更利于后期功能的扩展以及问题的排查，而且可以减少复杂的业务场景下数据同步的成本。

2.4 ANT DESIGN

Ant Design 给系统提供比较完整的界面组件基础，有表单、按钮、表格、抽屉、弹窗、标签、列表、统计卡片等常用的组件。依靠这些成熟的组件，平台可以很快创建出需求录入、任务展示、后台审核、数据看板等页面，并且可以统一控制适老化界面的字号、间距、反馈方式。

在本项目中，Ant Design 的使用一方面可以提高页面搭建的速度，另一方面也可以保证各个角色界面风格的一致性。老年人或者家属端、志愿者端和管理员端虽然关注点不同，但是都需要有清晰、规范、容易操作的交互逻辑。使用表单验证、表格显示、消息提示、弹窗反馈等成熟的组件之后，开发过程就可以更多地集中于业务逻辑上。基于组件库进行适老化调整，有利于对字号、按钮大小、留白以及交互反馈等各方面进行统一控制。

2.5 SOCKET.IO

Socket.io 用作平台的实时通信。社区助老服务场景里，订单接单、状态变动、过程证实、双端聊天都具备较强实时性需求，于是项目采用 Socket.io 来创建订单级聊天室以及消息推送，从而改善服务协同速度。

根据本项目业务场景，Socket.io 的价值在于“即时同步”。当订单被接单、审核通过、状态变更或者聊天消息发送的时候，系统可以利用长连接的方式把结果推送给对应的客户端，防止用户频繁刷新页面来获取最新的状态。社区助老服务属于跨角色协同性较高的应用范畴，它会使得实时通信得以缩短信息传递链路，并且可以提高服务执行过程中反馈的速度以及用户体验，进而使平台在需求确认、服务跟进以及异常沟通等方面更为流畅。

2.6 MYSQL 与 REDIS

MySQL 一般用作存储用户的详细信息、老人的档案资料、服务的订单详情、聊天时长、积分的明细账、通知的公告内容等结构化业务数据，Redis 则经常被用来存储验证码、热点数据的缓存、部分临时的状态等数据。两者结合可以满足核心业务数据的持久化需求，也可以提高平台的访问效率。

本项目架构当中，MySQL 担当起主要业务数据长久保存的任务，可以处理用户账号，老人档案，服务订单，聊天记录以及积分流水这些带有明显结构联系的数据对象。Redis 适合于处理访问次数多、时效性短的缓存数据和临时状态，

验证码、登录校验信息以及部分热点查询结果等都属于此类数据。两者配合使用，一方面可以保证业务数据的完整、可追溯，另一方面也可以通过缓存机制提高系统高并发时的响应速度，给平台以后的扩展提供更好的数据支持。

2.7 高德地图 API 与 ECHARTS

高德地图 API 可以完成地图选点、任务定位、志愿者任务分布展示等功能，使得用户可以正确填写服务地点，也可以通过距离和路线来判断自己是否可以接单。ECharts 用以管理员端的数据可视化展示，把需求类型、订单完成率、活跃用户、热点区域等信息转成图表。

高德地图 API 在本平台里除了完成位置展示的功能之外，还会参与到需求发布以及任务筛选过程中有关的空间信息处理当中。利用地图选点、坐标解析、位置回显等手段，使用户可以更清楚地知道服务地点，从而降低由于文字地址表述不准确所造成的沟通成本。志愿者端地图能力可以与距离、路线、区域分布相结合来判断接单是否可行。ECharts 主要是为管理员端提供数据统计分析服务，将订单数量、需求类型、服务完成率、用户活跃度、热点区域分布等信息用图形化的形式展现出来，有利于管理人员对平台运行状况有整体的认识并提高决策的效率。

第3章 系统需求分析

3.1 可行性分析

可行性分析目的在于证明社区助老服务对接平台在目前的技术状况、实际使用环境以及建设成本限制之下，是否具有实施的基础。根据本项目技术选型、业务范围以及已有的实现情况来判断，该平台有较好的落地条件。社区助老服务本身就存在着一定的现实需求，老年人、家属、志愿者和社区管理人员之间存在着不断的接触和服务关系，因此平台建设不是脱离实际场景的理论设计，而是以解决具体问题为出发点的信息化整合。经过对服务发布、人员匹配、过程跟踪、结果反馈、后台管理等主要环节的梳理，可以得出本项目有较强的实用性以及实施价值。本文从技术可行性、操作可行性和经济可行性三个方面进行分析。

3.1.1 技术可行性

React、Node.js、Express、MySQL、Redis、Socket.io、高德地图 API 和 ECharts 等技术均较为成熟，能够覆盖本项目在前端交互、后端服务、数据存储、实时通信、地图展示和统计分析方面的开发需求。React 适合创建多角色、多页面、多组件协同的前端界面，可以很好地支持老年人、家属端、志愿者端、管理员端在交互逻辑上不同的设计；Node.js 和 Express 适合搭建接口清晰、响应及时的服务端结构，方便实现登录认证、需求发布、订单流转、审核管理、消息通信等功能。MySQL 用来保存用户的个人信息、老人的档案、订单数据、聊天记录以及积分信息等主要业务数据，Redis 用来承担验证码、缓存数据、临时状态的存储工作，从而提高系统的响应速度。

Socket.io 能够给平台赋予即时聊天、状态同步等特性，契合助老服务场景里消息及时传达、订单进展及时反馈的需求，高德地图 API 可以实现任务地图展示、服务地点选择、位置筛选等功能，使志愿者接单、用户发布需求更直观，ECharts 则把订单数量、需求类型、活跃用户、服务完成率等数据以可视化的形式表现出来，加强了后台数据的分析水平。根据现有项目代码中包含的多角色页面、聊天组件、积分商城、任务地图、后台管理系统可以推断出该技术路线是可行的。同时这些技术之间配合关系比较清楚，开发资料齐全、社区支持较好，有利于之后的调试、维护以及功能扩展，因此从技术角度来说，本项目在技术方面具有较好的实施可行性。

3.1.2 操作可行性

平台为老年人、家属、志愿者和管理员这四类用户提供不同的界面。老年人、家属端以简洁的入口、明确的反馈为主，志愿者端以任务浏览、服务执行为主，

管理员端以审核、调度、统计分析为主，分角色设计可以降低学习成本，符合社区助老场景下实际使用习惯。对于老年用户来说，平台的设计应该尽量简化复杂的操作路径，避免出现信息堆积、按钮分散、术语难懂等状况，用更加清晰的功能入口、更加直观的页面布局、更加直接的提示信息来提高用户对系统功能的理解程度。家属用户比老年人对数字设备的操作能力强，可以在注册登录、档案管理、订单查看、服务沟通等环节协助老年人完成操作，所以平台在家庭协同使用方面具有较好的可行性。

对于志愿者用户来说，平台的主要使用流程是任务查看、条件筛选、接单执行、过程反馈、结果提交等，这些操作逻辑同常见的任务型应用比较接近，理解难度较小。管理员用户主要负责用户审核、需求审核、订单调度、公告发布、统计分析、运行维护等管理工作，后台界面功能结构比较清晰，有利于日常管理工作。从实际应用角度来讲，社区养老服务本来就需要多主体协同配合，平台的出现正好可以把原本分散在电话、微信群、纸质登记等线下、线上各个地方的信息集中到一个统一的操作入口上来。因此，只要界面设计保持简洁明了、反馈及时、流程清楚，本系统在操作上就有推广和使用的条件。

3.1.3 经济可行性

本项目使用开源框架、常用开发工具进行开发、部署，开发、部署成本可控。平台上线之后，在一定程度上减少了社区人工沟通、纸质记录、重复协调的成本，提高了资源匹配的效率以及管理的透明度，因此有现实建设的价值。React、Node.js、Express、MySQL、Redis 等主要的开发技术可以在常规开发环境中使用，不需要购买昂贵的软件授权，从而减少了前期的开发成本。同时系统采用前后端分离架构，利于后期按模块维护、逐步扩展，当需求增加时可以采用增量开发的方式进行功能的补充，不会造成一次性投入过大而产生的成本压力。

就使用收益而言，本平台可以促使社区更好地整合起助老服务资源，缩减响应时间，削减由于沟通滞后，记载不够齐全或者流程不明朗造成的管理开支。对老年人和家属而言，平台可以提高服务获取的效率和过程的可见性；对志愿者而言，平台可以提高任务分配的效率以及参与的便利性；对社区管理人员而言，平台可以提高数据沉淀的能力以及运营管理能力。尽管系统建设之初仍然要投入一定的人员去开展需求梳理、功能开发、测试部署以及使用培训工作，但是总体上讲，由于系统所带来服务优化所带来的效益远大于系统的成本，因此是合乎理性的。所以从经济投入和实际收益的综合考虑来说，本项目有较好的经济可行性。

3.2 功能需求分析

平台功能需求可以分为老年人/家属端、志愿者端、管理员端三个层次。老

年人/家属端支持注册登录、老人档案管理、需求发布、订单查询、实时聊天、结果评价、个人信息管理，志愿者端支持任务大厅浏览、地图筛选、接单执行、过程反馈、积分累计和兑换，管理员端支持用户审核、需求审核、订单调度、公告发布、统计分析和平台运营管理。经过上述功能的划分，系统可以满足社区助老服务中提出的需求、进行任务匹配、执行服务、反馈结果、后台管理等各个环节，形成一个比较完整的业务闭环。

老年人/家属端除了要完成基础信息填写和服务申请提交之外，还要体现需求发布过程的简化设计，即用清晰的分类入口、地点选择方式、时间选择提示、内容填写说明等来帮助用户准确地表达出自己的服务需求。系统还要能够查看历史订单、跟踪当下的状态并和志愿者及时联系，从而提高服务过程的透明度和可追踪性。志愿者端除了任务浏览、接单等功能之外，还需要具备与服务位置有关的地图辅助功能，让用户可以依据距离、区域、任务类型等信息来判断接单是否可行，过程反馈、完成确认、积分统计等功能也十分重要，一方面影响着服务执行记录的完整程度，另一方面也影响着志愿者参与积极性的不断提升。

管理员端是整个平台稳定运行的保证。管理员除了要对注册用户、志愿者身份、服务需求、订单状态进行审核和调度外，还要对公告通知、平台数据、服务完成情况、异常订单进行统一管理。后台统计分析功能可以对不同时段的需求分布、常用的服务种类、活跃的志愿者数量以及订单完成率等进行统计，为之后社区资源调配和服务改进提供数据支撑。除了功能性的需求之外，平台还需要满足界面的易用性、业务的实时性、系统的可扩展性以及数据的安全性这些非功能性的需求，在适老化的场景之下，对页面的理解成本和操作的复杂程度加以控制。同时考虑到平台以后会接入更多的社区服务内容，比如健康提醒、紧急联系、活动报名、社区公告联动等，系统的架构设计也要留有扩展的空间，以便于以后的升级。如表 3-1 所示。

表 3-1 功能需求分析表

功能层次	主要功能	功能说明
老人/家属端	注册登录、老人档案、需求发布、订单查询、实时聊天、结果评价	突出需求表达清晰、操作简洁与服务过程可追踪，满足老年用户及家属的使用场景。
志愿者端	任务浏览、地图筛选、接单执行、过程反馈、积分累计与兑换	突出接单效率、地图辅助和服务反馈，兼顾志愿服务记录与激励机制。
管理员端	用户审核、需求审核、订单调度、统计分析、平台运营管理	负责平台审核、异常处理与资源调配，保障业务流程稳定运行。
共享支撑功能	实时通信、状态同步、数据统计、安全控制	为多角色协同提供基础能力支撑，提升沟通效率与系统稳定性。

3.3 业务流程分析

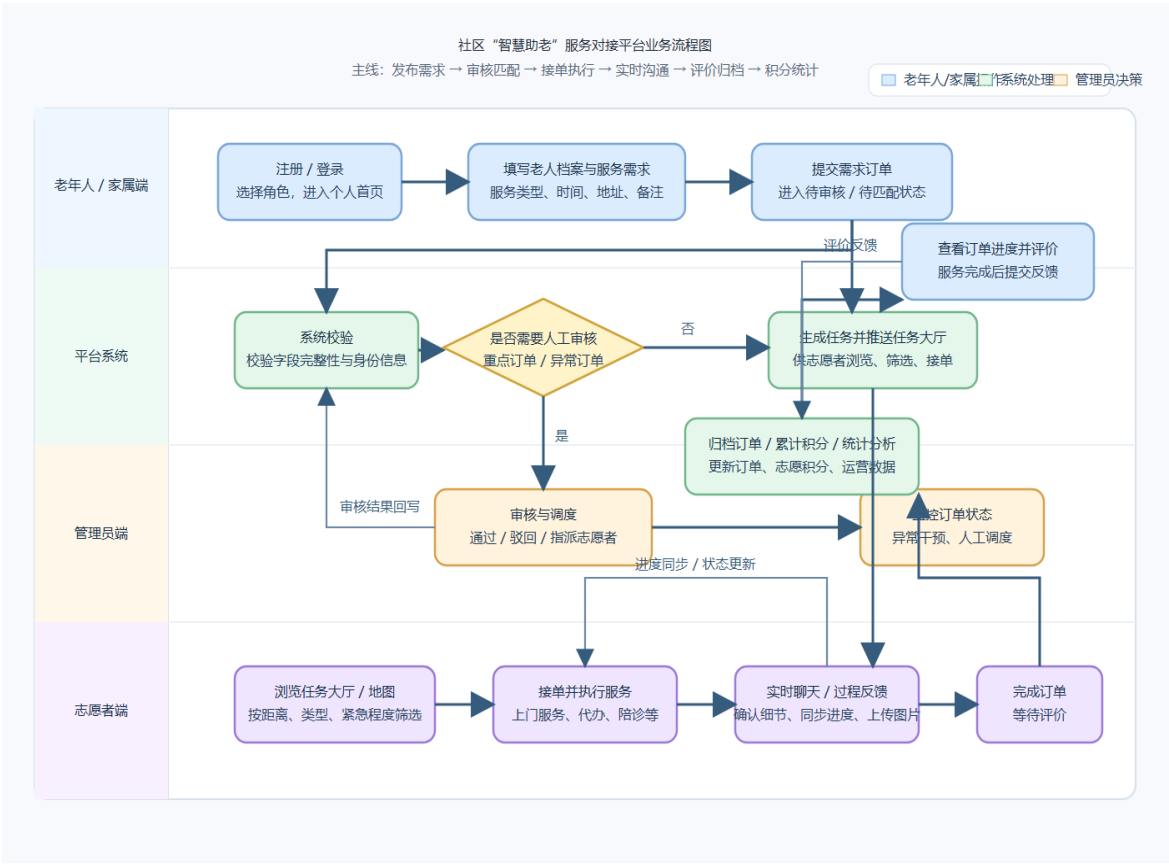


图 3-1 业务流程图

平台业务流程是以发布需求、审核匹配、接单执行、结果反馈、归档评价为

一条主线来开展的。老年人或者家属先提出服务需求，系统做基础校验之后产生服务订单；管理员可以对重点或者异常订单展开审核干预；志愿者在任务大厅里浏览并接单；服务执行期间，双方可以借助实时聊天来确定细节并同步状态；服务结束之后，老年人或者家属给出评价，系统保存积分以及统计成果。如上图 3-1 所示。

3.4 数据流程分析

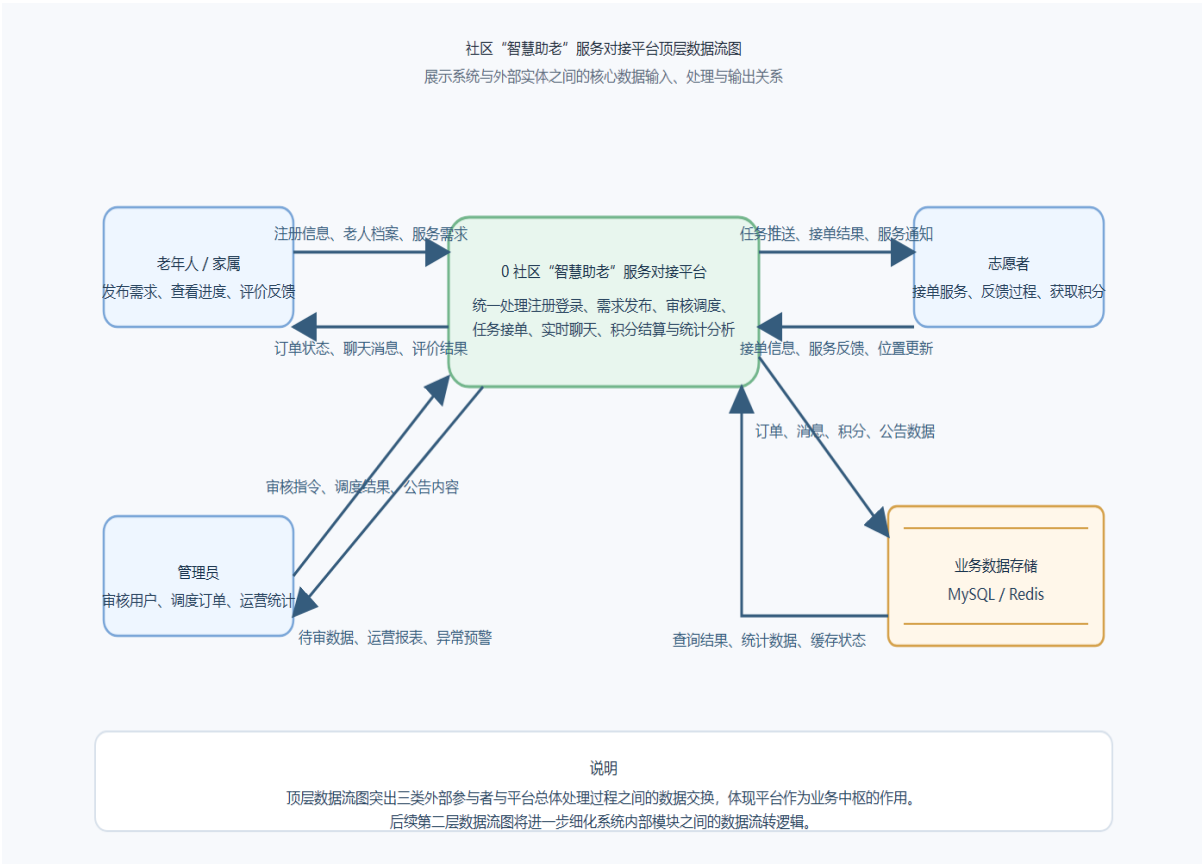


图 3-2 顶层数据流图

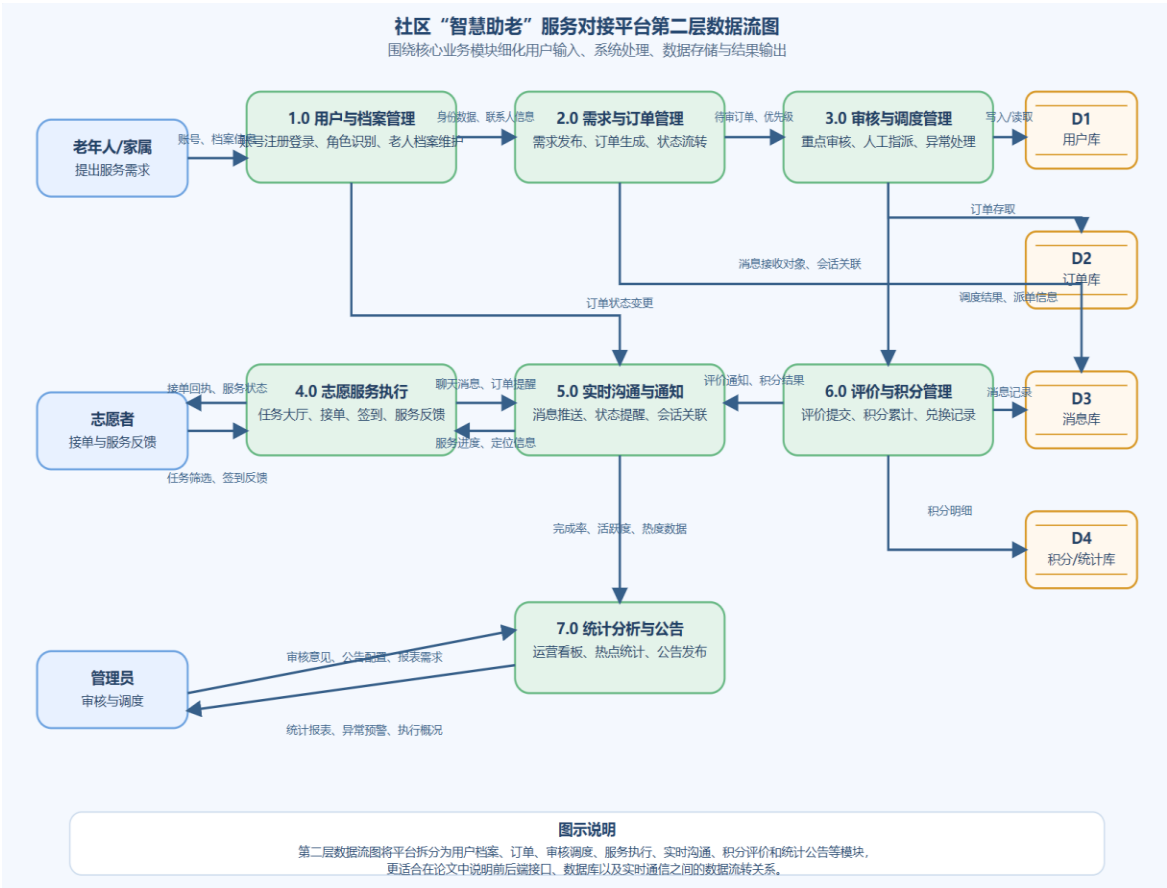


图 3-3 第二层数据流图

从数据流程上来说，平台的数据流起始点是用户在前端页面上的输入行为，即注册信息、老人档案、服务需求、聊天消息、评价内容等。前端页面将数据提交到后端接口之后，系统会对身份进行验证、字段进行校验、状态进行判断、持久化处理；然后相关数据会被写入 MySQL 或者缓存到 Redis 中，在需要的时候通过 Socket.io 推送给相应的用户端；最后将处理结果返回给前端页面，供订单展示、消息同步、统计分析使用。如上图 3-1 和图 3-2 所示。

第 4 章 系统详细设计

4.1 系统架构

本系统使用前后端分离架构。前端使用 **React** 创建多角色业务界面，用户交互、页面渲染、状态展示；后端使用 **Node.js+Express** 搭建服务接口，业务处理、权限控制、数据读写；**MySQL** 用来保存核心业务数据，**Redis** 做缓存以及临时状态的管理。整体架构清楚地将系统分为表现层、业务层、数据层，有利于系统的维护以及功能的扩展。

在表现层设计中，根据老年人、家属、志愿者、管理员这四种用户不同的使用需求来创建不同的界面。前端采用组件化的形式对页面结构进行组织，把登录注册、老人档案、需求表单、任务列表、订单详情、聊天窗口、统计面板等拆分成相对独立的界面单元，从而提高页面的复用性以及后续的维护效率。由于社区智慧助老服务平台要实现多角色切换、多业务入口、多状态展示，所以采用分层组织页面、分层组织组件的方式更为有利于系统的运行。

后端主要是处理系统主要业务逻辑部分，即用户认证、角色权限校验、服务需求发布、订单状态流转、消息传递、积分累积、后台审核管理等。**Node.js** 异步处理机制可以很好地适应平台中高频接口调用和实时交互并存的业务特点，**Express** 通过路由、中间件、控制器等方式把不同的模块拆分出来，使得用户管理、订单管理、审核管理、统计管理等功能具有较好的组织性。该种设计可以提高后端代码的可读性、可维护性，也可以方便以后根据业务的发展对单个模块进行修改和扩展。

数据层的设计上，**MySQL** 主要负责平台核心业务数据的长期保存工作，即用户的个人信息、老人的档案资料、服务的需求情况、订单的信息、聊天的内容、积分的流水以及公告的消息等所有结构化的数据都被存储到 **MySQL** 里；**Redis** 被用来缓存热点数据、保存验证码、临时会话状态等，从而减轻数据库的压力，加快系统的反应速度。对于实时性的高要求聊天消息、状态变化等用 **socket.io** 推送消息，同步到界面来提升平台交互的即时性。从整体上看，该架构可以满足目前系统开发和运行的要求，并且给以后增加新的功能模块、提高性能、接入更多的社区服务场景留有较好的扩展余地。

4.2 系统功能模块设计

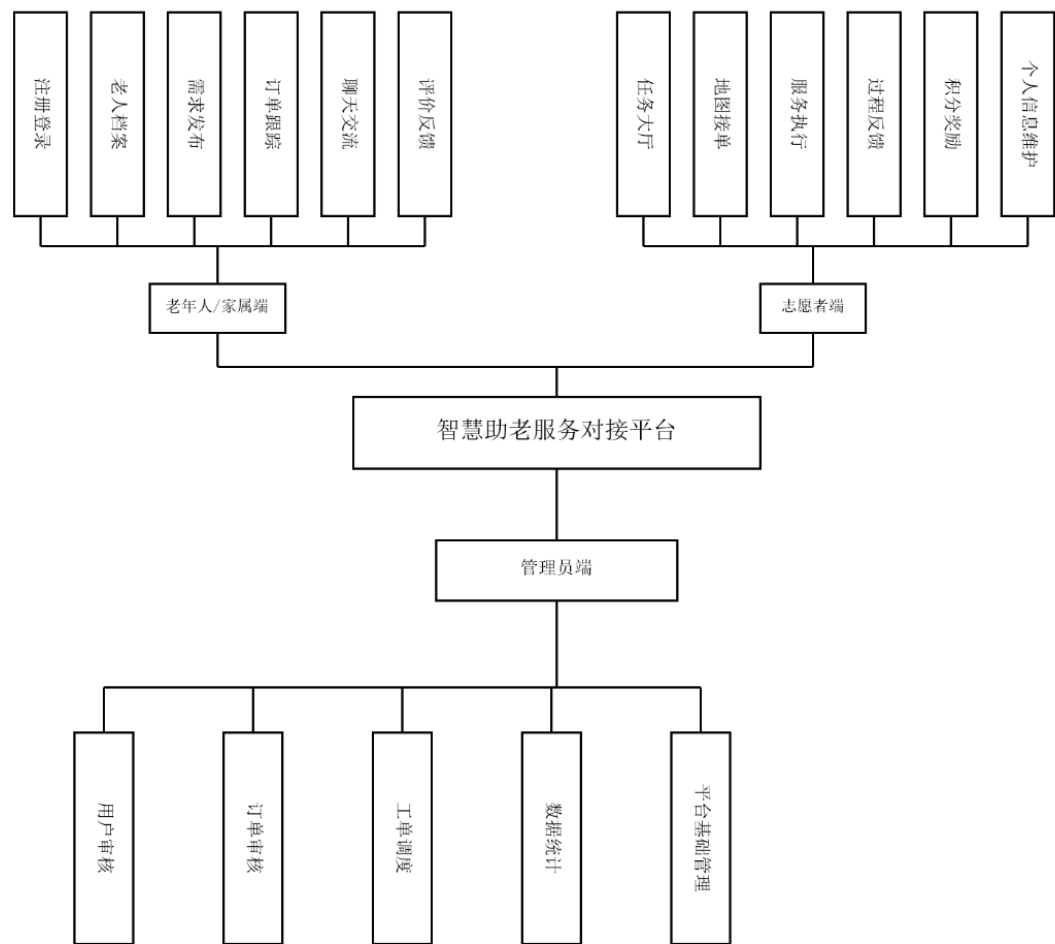


图 4-1 系统模块功能设计图

系统功能模块设计是以三类核心用户为对象的。老年人家属端有注册登录、老人档案、需求发布、订单跟踪、聊天交流、评价反馈，志愿者端有任务大厅、地图接单、服务执行、过程反馈、积分奖励、个人信息维护，管理员端有用户审核、订单审核、工单调度、数据统计、平台基础管理。

老年人/家属端为平台服务需求的来源，所以在模块设计上更加注重操作简便、流程明了、反馈清楚。注册登录模块完成用户身份的创建以及访问控制；老人档案模块保存老人的基本信息、联系方式、居住地点、特殊说明等资料，给后面的服务匹配提供基础的数据支持；需求发布模块用来收集服务类型、时间、地点、详细描述等信息，使用户可以较为完整地表达自己的需求。订单跟踪模块主要是对需求受理情况、志愿者接单状态和服务执行进度进行查看，聊天沟通模块是服务前、中、后与志愿者进行信息交流的工具，评价反馈模块是服务结束后记录用户满意度和文字评价的平台，可以形成比较完整的闭环服务过程。

志愿者端模块的设计更注重任务发现、任务执行、参与激励这三个方面。任

务大厅模块对外部所需求的服务予以集中呈现，使志愿者能即时找到正处在处理中的任务，地图接单模块融合了位置显示及区域挑选功能，让用户依据距离、路线和服务散布状况来决定是否接单，服务执行模块聚焦于接单之后的具体业务操作，涵盖查看订单详情、核对服务信息并推动服务进程等内容，过程回馈模块在服务执行期间提交阶段性的说明，加强服务进程的可看性，积分奖励模块存档并累积用户每一次的服务行为，为今后的积分交换或者荣誉激励做好铺垫，个人信息维护模块可对用户的个人信息、服务记录、账号基本资料等加以管理，提升用户长时间使用时的体验。

管理员端模块是平台正常运转的保证，其功能设计更加重视审核控制、资源调度、运营分析。用户审核模块主要是核验注册信息以及志愿者身份，保证平台参与主体的真实性以及规范性，订单审核模块是对服务需求内容进行检查，防止无效、重复或者不规范的订单直接进入业务流转，工单调度模块在任务无人接单、信息异常或者服务冲突的时候提供人工干预的途径，从而提高整个业务处理的效率。数据统计模块可以对订单数量、需求类型、用户活跃度、服务完成率等主要指标进行展示，平台基础管理模块包括权限维护、基础配置、运行状态管理等。经过以上模块的配合使用，可以较好地支持社区助老服务从需求提出到服务执行再到后台监管的全部过程。如上图 4-1 所示。

4.3 系统用户设计

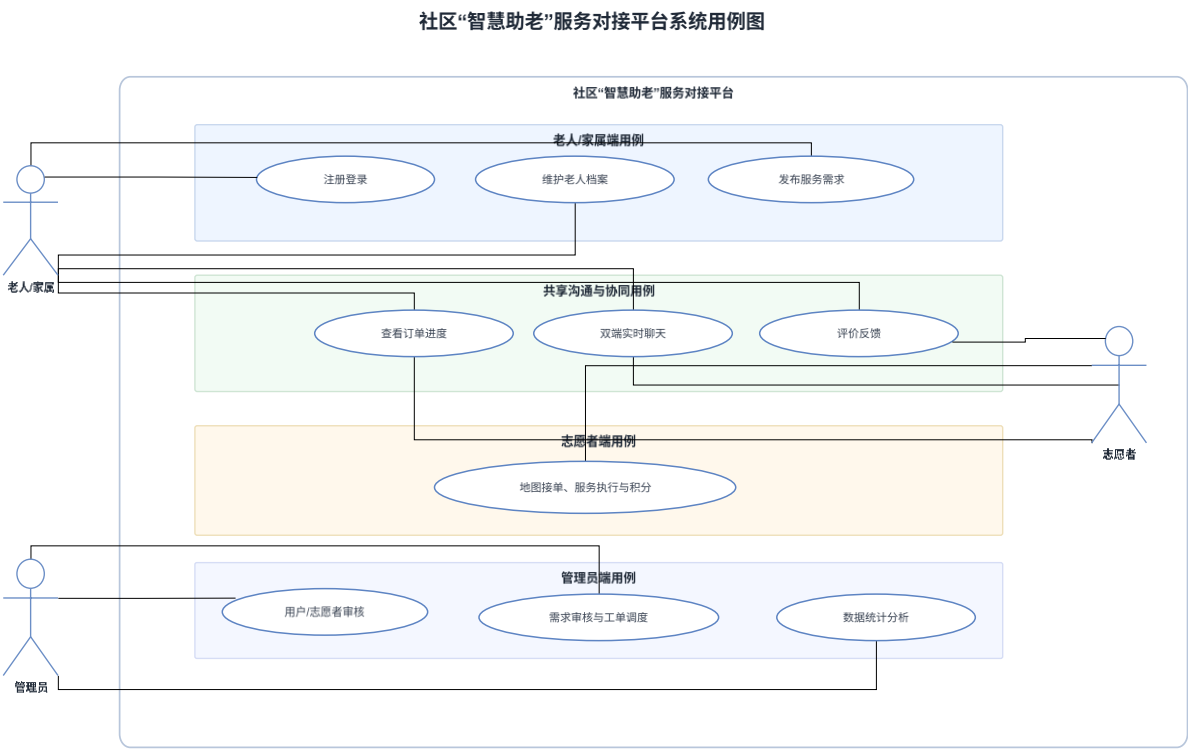


图 4-2 系统用例图

从用户设计的角度来说，平台包含老年人、家属、志愿者、管理员这四种主要的使用者。老年人或者其家属提出需求之后，跟踪结果，志愿者浏览任务并执行服务获取积分，管理员通过平台审核、调度、运营管理以及统计分析。三类角色共用一套核心业务数据，但是权限、界面有明确的界限。如上图 4-2 所示。

4.4 数据库设计

4.4.1 概念模型设计

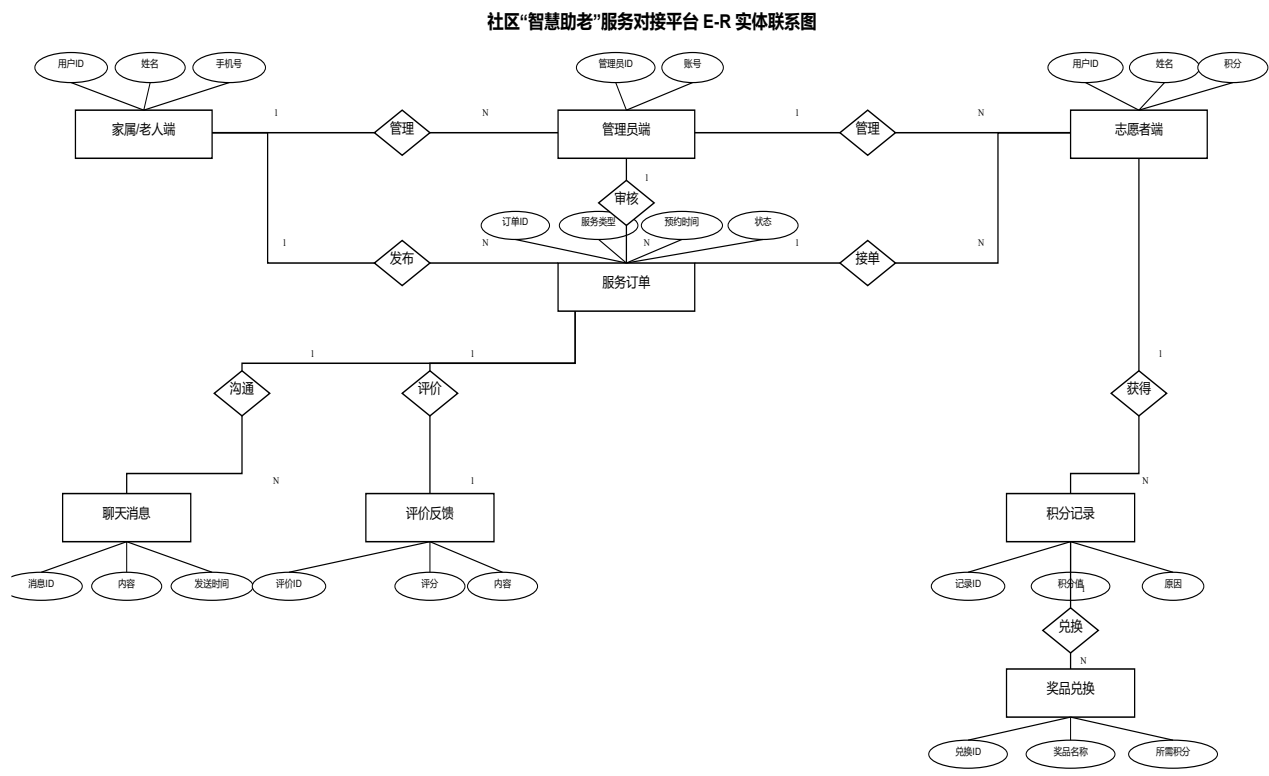


图 4-3 E-R 实体联系图

数据库概念模型是以用户、老人档案、服务订单、聊天消息、积分流水这五个主要的实体为中心来构建的。用户实体保存平台账号和角色信息，老人档案实体保存被服务老人的基本信息和联系人信息，服务订单实体保存需求内容、时间、地点、状态和服务执行过程，聊天消息实体保存围绕订单产生的实时沟通内容，积分流水实体保存志愿者积分的增加、扣减和兑换情况。如上表 4-3 所示。

4. 4. 2 数据库表设计

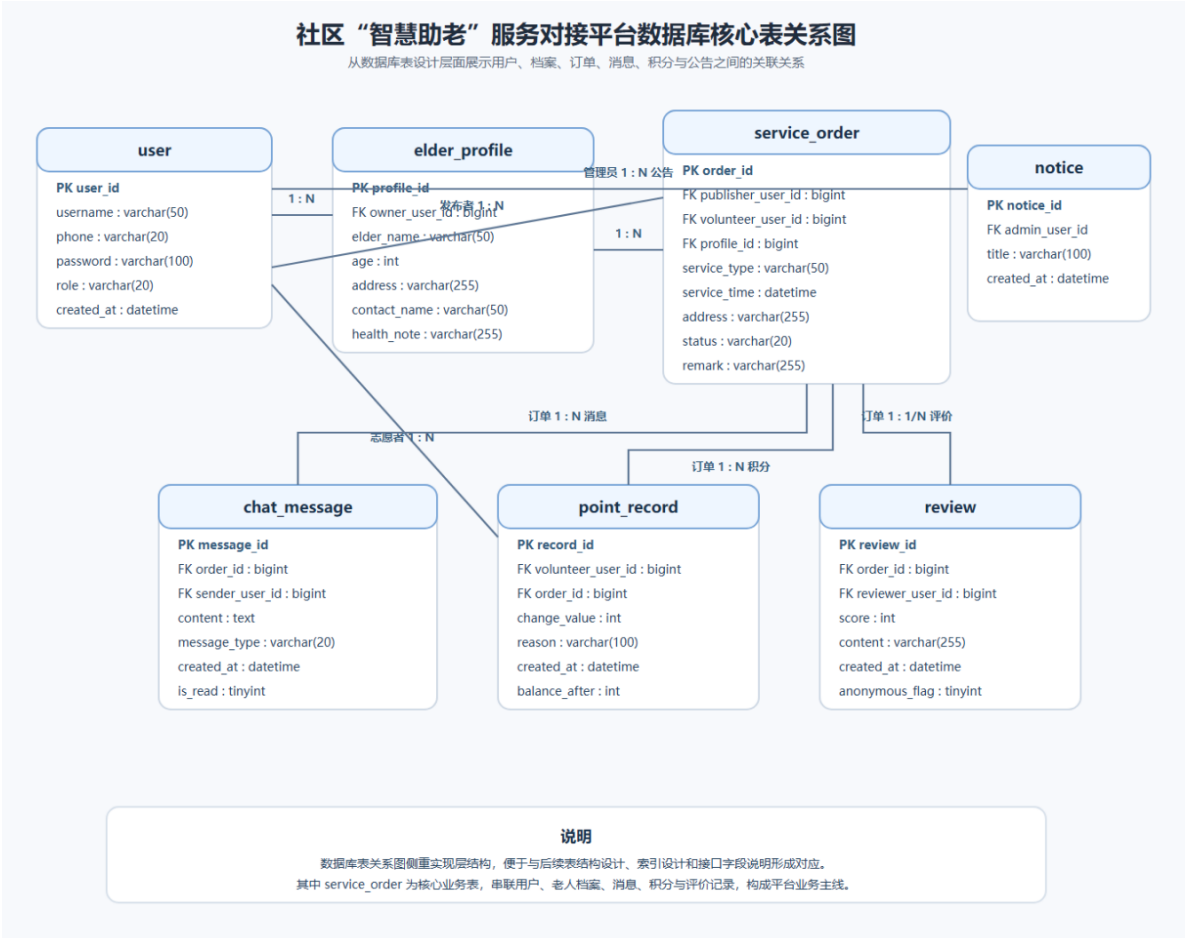


图 4-4 数据库核心表关系图

系统在数据库表的设计上主要是用户表、老人档案表、服务订单表、聊天消息表、积分流水表、公告表等主要的数据库表。用户表保存账号、角色、联系方式，老人档案表保存老人姓名、年龄、地址、联系人、健康备注，服务订单表保存需求类型、服务时间、服务地点、订单状态、发布人、接单志愿者等信息，聊天消息表保存消息内容、发送者、时间戳，积分流水表保存积分变化值、变化原因、关联订单。如上图 4-4 所示。

表 4-1 用户表（user）

字段名	类型	键	是否为空	说明
user_id	bigint	PK	否	用户主键编号
username	varchar(50)		否	用户名
phone	varchar(20)		否	手机号，用于登录
password	varchar(100)		否	加密后的登录密码
role	varchar(20)		否	角色：elder/volunteer/admin
created_at	datetime		否	创建时间

用户表主要存储平台登录账户的基础信息，`user_id` 为主键，`phone` 字段用于登录识别，`password` 字段保存加密后的密码，`role` 字段采用 `varchar(20)`保存角色枚举值，便于区分家属、志愿者和管理员三类用户。如上表 4-1 所示。

表 4-2 老人档案表 (elder_profile)

字段名	类型	键	是否为空	说明
profile_id	bigint	PK	否	老人档案主键
owner_user_id	bigint	FK	否	所属家属用户 ID
elder_name	varchar(50)		否	老人姓名
age	int		是	年龄
address	varchar(255)		否	服务地址
contact_name	varchar(50)		否	联系人姓名
health_note	varchar(255)		是	健康备注

老人档案表用于记录服务对象的核心资料，其中 `owner_user_id` 关联家属用户，`age` 和 `health_note` 允许为空，以适配部分信息暂未完善的场景；`address`、`contact_name` 等字段为后续服务派单与联系提供依据。如上表 4-2 所示。

表 4-3 服务订单表 (service_order)

字段名	类型	键	是否为空	说明
order_id	bigint	PK	否	服务订单主键
publisher_user_id	bigint	FK	否	需求发布人 ID
volunteer_user_id	bigint	FK	是	接单志愿者 ID
profile_id	bigint	FK	否	关联老人档案 ID
service_type	varchar(50)		否	服务类型
service_time	datetime		否	预约服务时间
address	varchar(255)		否	服务地点
status	varchar(20)		否	订单状态
remark	varchar(255)		是	备注信息

服务订单表用于描述一次养老服务需求的完整业务数据，`publisher_user_id`、`volunteer_user_id` 和 `profile_id` 分别关联发布人、接单志愿者及老人档案；其中 `volunteer_user_id` 可为空，表示订单尚未被接单，`status` 字段用于控制订单流转状态。如上表 4-3 所示。

表 4-4 聊天消息表 (chat_message)

字段名	类型	键	是否为空	说明
message_id	bigint	PK	否	消息主键
order_id	bigint	FK	否	关联订单 ID
sender_user_id	bigint	FK	否	发送者 ID
message_type	varchar(20)		否	消息类型
content	text		否	消息内容
is_read	tinyint		否	是否已读
created_at	datetime		否	发送时间

聊天消息表用于保存订单沟通过程中的消息记录，order_id 和 sender_user_id 分别关联所属订单与发送者；content 字段采用 text 类型以兼容较长文本内容，is_read 使用 tinyint 标识已读状态，便于实现消息提醒与未读统计。如上表 4-4 所示。

表 4-5 积分流水表 (point_record)

字段名	类型	键	是否为空	说明
record_id	bigint	PK	否	积分流水主键
volunteer_user_id	bigint	FK	否	志愿者 ID
order_id	bigint	FK	是	关联订单 ID
change_value	int		否	积分变化值
reason	varchar(100)		否	积分变化原因
balance_after	int		是	变更后余额
created_at	datetime		否	记录时间

积分流水表用于记录志愿者积分变动情况，change_value 表示本次增减分值，reason 用于说明积分来源，balance_after 可为空以兼容部分历史记录或延迟结算场景；当积分来源于服务订单时，可通过 order_id 进行追溯。如上表 4-5 所示。

表 4-6 关键状态数据字典

字段	取值示例	说明
user.role	elder / volunteer / admin	分别表示家属端、志愿者端和管理员端角色
service_order.status	pending / approved / assigned / in_service / completed / cancelled	订单从发布到完成的主要流转状态
chat_message.message_type	text / image / system	聊天消息类别

续表 4-6

字段	取值示例	说明
notice.status	draft / published / offline	公告草稿、已发布和下线状态
point_record.reason	order_complete / bonus / exchange	积分增加、奖励或兑换来源
review.score	1-5	服务评价星级

关键状态数据字典集中列出了系统中的主要枚举字段及典型取值，用于统一角色、订单状态、消息类型等业务语义；其中各字段均以字符型状态码保存，实际开发中应保持前后端取值一致，避免状态判断歧义。如上表 4-6 所示。

第 5 章 系统实现

5.1 登录界面

智慧助老

社区互助服务对接平台

请输入手机号或账号

请输入密码 (最长12位)

登录

没有账号? [立即注册](#)

欢迎注册

加入智慧助老大家庭

* 您是?

需要帮助 (老人/家属)

提供帮助 (志愿者)

手机号 (11位数字, 作为登录账号)

设置密码 (最长12位)

真实姓名

身份证号码

注册

已有账号? [返回登录](#)

图 5-1 登录界面

如图 5-1 所示，系统登录界面采用前后端分离的实现方式，前端通过表单组件收集手机号、密码等登录信息，并结合输入校验机制对必填项、格式合法性和

密码长度进行实时检查，减少用户误操作。后端在接收到请求后完成身份认证、角色识别和会话信息生成，再根据用户类型将其引导至老年人/家属端或志愿者端的功能页面。该界面整体设计遵循简洁直观的原则，在保证操作易用性的同时兼顾账户安全，为后续需求发布、接单服务和后台管理提供统一的访问入口。

其中实现 JWT 身份验证逻辑的代码如下：

代码清单 5-1 JWT 身份验证关键代码

```
// server/src/controllers/authController.ts

84     // Check if password matches
85     const isMatch = await bcrypt.compare(String(password), String(userData.password));
86     if (!isMatch) {
87         res.status(400).json({ message: '手机号或密码错误' });
88         return;
89     }
90
91     // Check status
92     if (userData.status === 'disabled') {
93         res.status(403).json({ message: '账号已被禁用，请联系管理员' });
94         return;
95     }
96
97     // Sign JWT synchronously
98     const token = jwt.sign(
99         { user: { id: userData.id, role: userData.role, status: userData.status } },
100         JWT_SECRET,
101         { expiresIn: '7d' }
102     );
103
104     res.json({
105         token,
106         user: {
107             id: userData.id,
108             phone: userData.phone,
109             role: userData.role,
110             name: userData.name,
111             points: userData.points,
112             status: userData.status
113         }
114     });
115 }

// server/src/middleware/authMiddleware.ts

7     user?: any;
```

```
8  }
9
10 export const auth = (req: AuthRequest, res: Response, next: NextFunction) => {
11   // Get token from header
12   const authHeader = req.header('Authorization');
13   if (!authHeader || !authHeader.startsWith('Bearer ')) {
14     return res.status(401).json({ message: 'No token, authorization denied' });
15   }
16
17   const token = authHeader.split(' ')[1];
18
19   try {
20     const decoded = jwt.verify(token, JWT_SECRET);
21     req.user = (decoded as any).user;
22     next();
23   } catch (err) {
24     res.status(401).json({ message: 'Token is not valid' });
25   }
```

5.2 老年人/家属端首页

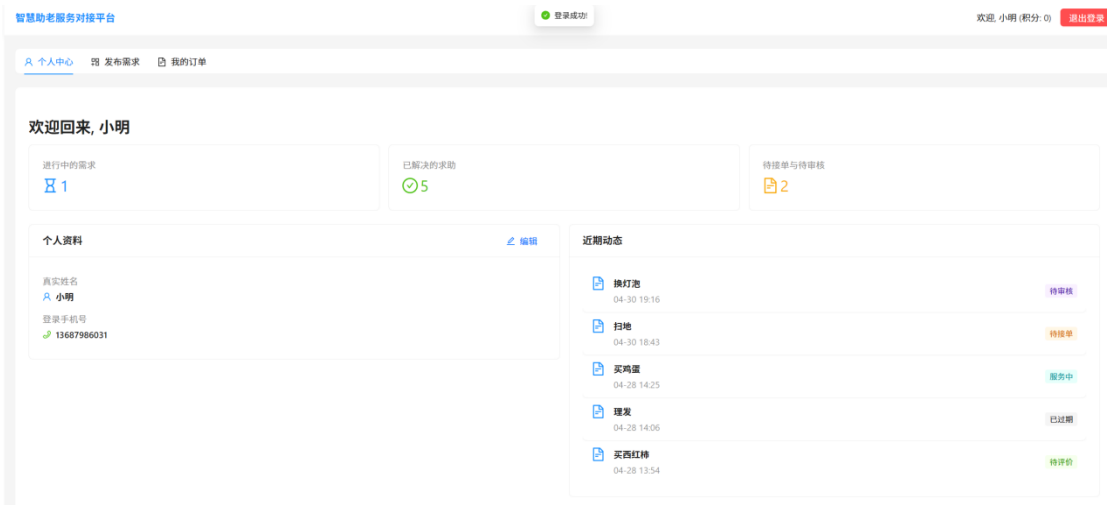


图 5-2 老年人/家属端首页

如图 5-2 所示,老年人/家属端首页主要承担平台核心功能入口的聚合作用,将需求发布、订单查看、消息沟通、个人信息维护等常用模块集中呈现,便于用户快速完成主要操作。页面布局上采用较为清晰的分区式设计,使重要功能保持可见,并通过卡片、按钮和状态提示等方式增强界面可读性,降低老年用户的学习成本。从实现原理上看,首页通常通过接口获取用户基本资料、服务状态和消息摘要等数据,再以组件化方式完成动态渲染,从而保证页面内容能够随着业务

状态变化及时更新。

其中实现首页数据加载与状态统计的关键代码如下：

代码清单 5-2 老年人/家属端首页关键代码

```
// client/src/pages/elderly/Dashboard.tsx
75     }, []);
76
77     const fetchOrders = async () => {
78         try {
79             setLoading(true);
80             const res = await api.get('/orders');
81             // Sort by latest
82             const sortedOrders = res.data.orders.sort((a: Order, b: Order) =>
83                 new Date(b.createdAt).getTime() - new Date(a.createdAt).getTime()
84             );
85             setOrders(sortedOrders);
86         } catch (error) {
87             console.error(error);
88             message.error('获取订单数据失败');
89         } finally {
90             setLoading(false);
91         }
92     };
93
94     const fetchProfile = async () => {
129         const res = await api.get(`/orders/${id}`);
130         setSelectedOrder(res.data.order);
131     } catch (error) {
132         console.error(error);
133         message.error('获取订单详情失败');
134         setIsModalVisible(false);
135     } finally {
```


5.3 需求发布与订单跟踪







图 5-3 需求发布与订单跟踪

如图 5-3 所示,需求发布与订单跟踪模块实现了从需求录入到服务完成的全过程管理。用户在需求发布页面中可以填写服务类型、预约时间、服务地点及具体说明等信息,前端对表单内容进行完整性校验后提交至后端,后端再完成数据存储、工单生成和状态初始化等处理。订单跟踪页面则负责展示审核中、待接单、服务中、已完成等关键状态,并结合时间信息、服务详情和评价入口帮助用户持续了解业务进展。通过这一模块,平台把原本分散的服务申请、流转和反馈过程整合为清晰可追踪的业务链条,提高了服务过程的透明度和管理效率。

其中实现需求发布与订单创建逻辑的关键代码如下:

代码清单 5-3 需求发布与订单创建关键代码

```
// client/src/pages/elderly/PublishRequest.tsx

31      // Validation: Past time check
32      if (values.expectedTime.isBefore(dayjs())) {
33          Modal.warning({
34              title: '上门时间填写错误',
35              content: '您设置的期望上门时间不能早于当前时间。为了方便志愿者准时上门,请选择一个
未来的时间点哟!',
36              okText: '好的,我去修改'
37          });
38          return;
39      }
40
41      if (!selectedLocation) {
42          message.warning('请通过地图选择确切的服务地址以便志愿者导航');
43          return;
44      }
45      setLoading(true);
46      try {
47          await api.post('/orders', {
48              ...values,
49              expectedTime: values.expectedTime.toISOString(),
50              lat: selectedLocation.lat,
51              lng: selectedLocation.lng
52          });
53          message.success('需求发布成功!');
54          navigate('/elderly/orders');
55      } catch {
56          message.error('发布失败,请重试');
57      } finally {
58          setLoading(false);
```

```
59     }
60   };
61
62   const handleVoiceInput = async () => {
63     if (!isRecording) {
64
65     }
66   }
67 }
68
69 // server/src/controllers/orderController.ts
70
71 33   return plainOrders;
72 34 };
73 35
74 36 export const createOrder = async (req: AuthRequest, res: Response) => {
75 37   try {
76 38     const userId = req.user.id;
77 39     if (req.user.role !== 'elderly' && req.user.role !== 'admin') {
78 40       return res.status(403).json({ message: 'Only elderly/family or admin can publish
requests' });
79 41     }
80 42
81 43     const { title, category, description, address, expectedTime, lat, lng } = req.body;
82 44
83 45     const newOrder = await Order.create({
84 46       title,
85 47       category,
86 48       description,
87 49       address,
88 50       expectedTime,
89 51       lat,
90 52       lng,
91 53       elderlyId: userId,
92 54       status: 'under_review',
93 55       pointsReward: null
94 56     });
95 57
96 58     // We no longer notify volunteers here because it needs admin approval first
```

5.4 志愿者任务大厅

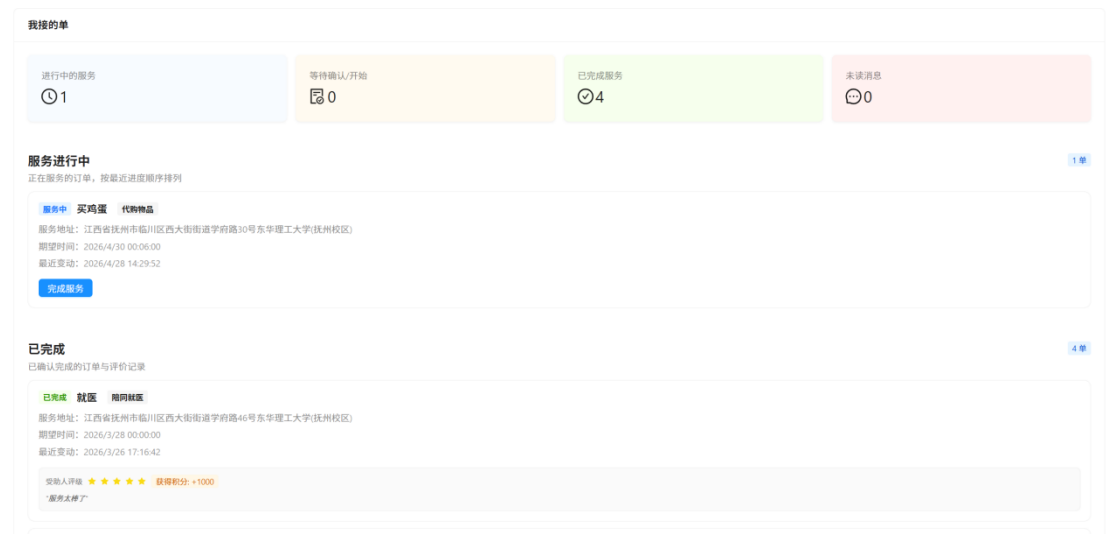


图 5-4 志愿者任务大厅

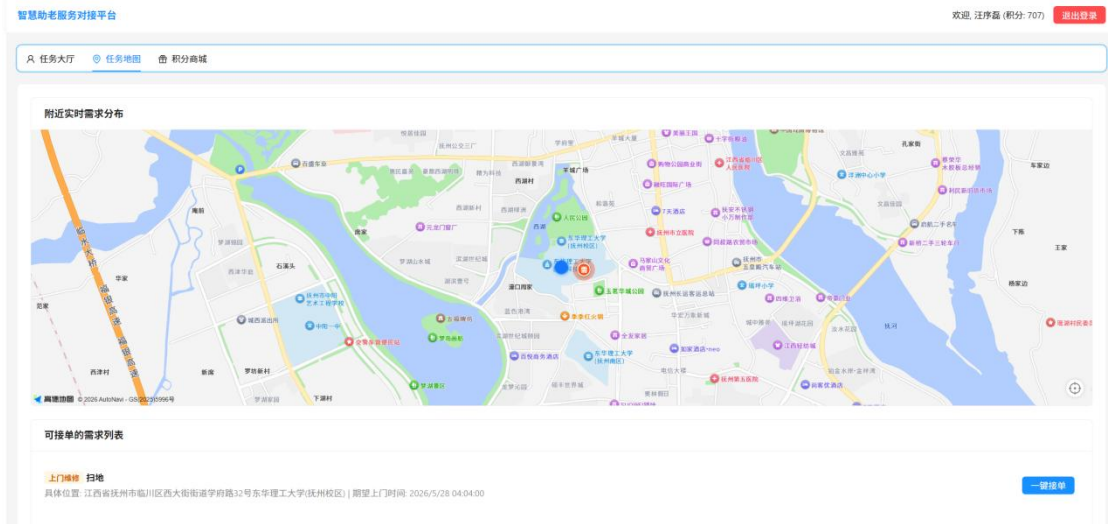


图 5-5 志愿者任务地图

如图 5-4 和图 5-5 所示，志愿者任务大厅分别以列表视图和地图视图展示当前可接单任务，便于志愿者从任务内容和空间位置两个维度进行综合判断。列表方式更适合快速浏览服务类型、发布时间和紧急程度等关键信息，地图方式则能够直观反映任务分布情况，帮助志愿者结合自身位置与出行成本选择更合适的服务对象。该模块在实现上需要完成任务数据查询、筛选条件联动以及地图定位展示等功能，并通过在线接单按钮将任务匹配与后续执行流程衔接起来，从而提升平台任务分配效率和响应速度。

其中实现任务地图加载、附近任务筛选与接单逻辑的关键代码如下：

代码清单 5-4 志愿者任务大厅关键代码

```
// client/src/pages/volunteer/TaskMap.tsx

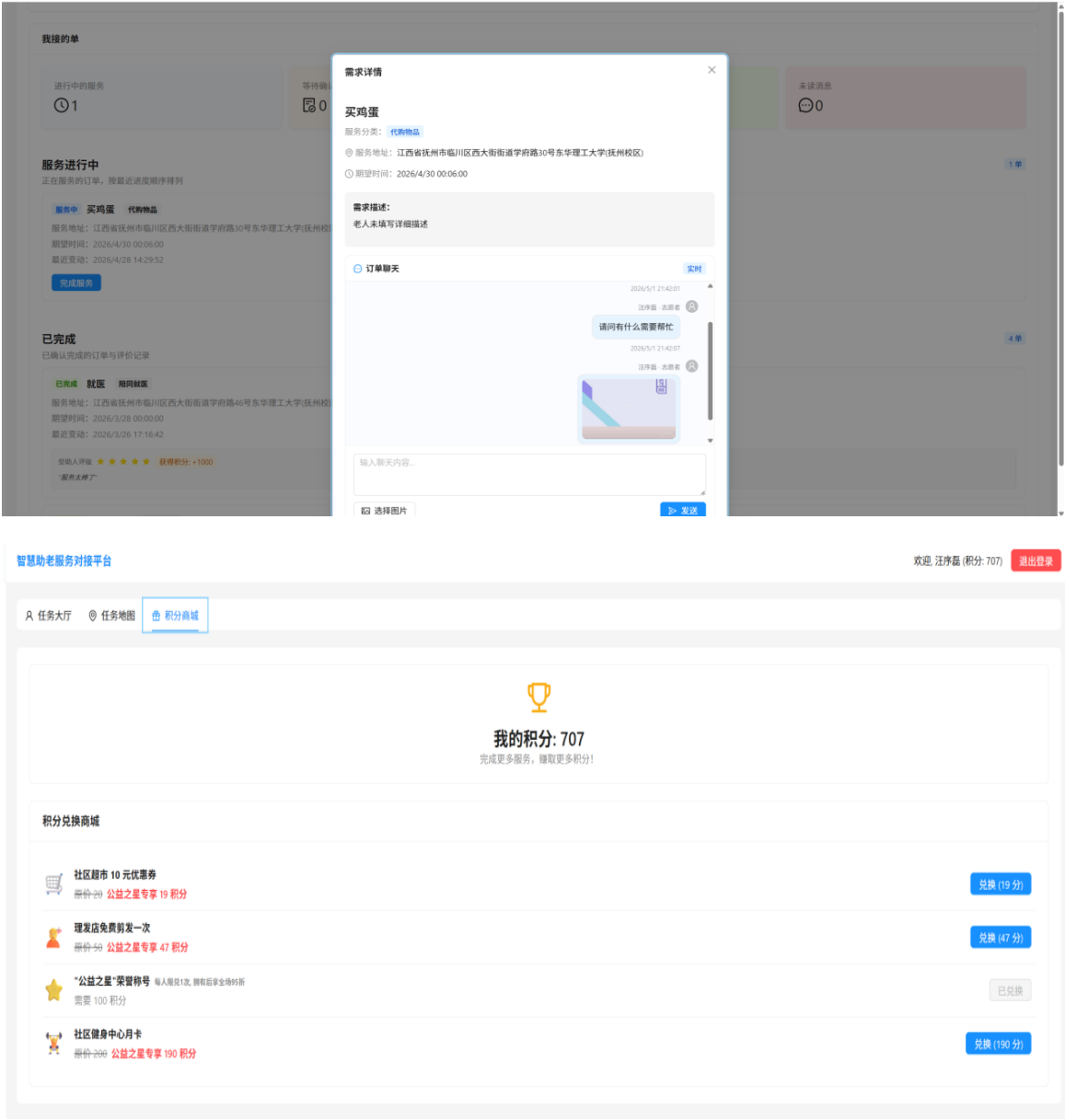
39      }
40    );
41  });
42 };
43
44 const TaskMap: React.FC = () => {
45   const mapRef = useRef<HTMLDivElement>(null);
46   const [loading, setLoading] = useState(false);
47   const [orders, setOrders] = useState<Order[]>([]);
48   const navigate = useNavigate();
49
50   const fetchOrders = useCallback(async (lat?: number, lng?: number) => {
51     try {
52       const params: Record<string, string | number> = { tab: 'available' };
53       if (lat !== undefined && lng !== undefined) {
54         params.lat = lat;
55         params.lng = lng;
56         params.radius = 15;
57       }
58
59       const { data } = await api.get('/orders', { params });
60       setOrders(data.orders);
61       return data.orders as Order[];
62     } catch {
63       message.error('无法获取待接单任务');
64       return [];
65     }
66   }, []);
67
68   const handleAcceptOrder = useCallback(async (orderId: number) => {
217       title: order.title,
218       content: customContent,
219       offset: new AMap.Pixel(-20, -20),
220     });
221
222     const contentHtml = `
223       <div style="padding: 10px; min-width: 250px;">
224         <h4 style="margin-top:0; margin-bottom: 8px; color:
#1890ff;">${order.title}</h4>
225         <p style="margin: 4px 0; font-size: 13px;"><strong>
```

```

分类:</strong> <span style="background: #e6f7ff; color: #1890ff; padding: 2px 6px; border-radius:
4px;">${order.category}</span></p>
226                                <p style="margin: 4px 0; font-size: 13px;"><strong>
地址:</strong> ${order.address}</p>
227                                <p style="margin: 4px 0; font-size: 13px;"><strong>
时间:</strong> ${new Date(order.expectedTime).toLocaleString()}</p>
228                                ${order.description ? `<p style="margin: 4px 0;
font-size: 13px; color: #666;"><strong>需求:</strong> ${order.description}</p>` : ''}
229                                <div style="margin-top: 12px; text-align: center;">
230                                    <button
231
onclick="window.acceptOrderFromMap(${order.id})"
232                                style="background: #1890ff; color: white;
border: none; padding: 6px 16px; border-radius: 4px; cursor: pointer; width: 100%;"
233                                >
234                                    一键接单
235                                </button>
236                                </div>
237                                </div>
238                                `;
239
240                                marker.on('click', () => {
241                                    infoWindow.setContent(contentHtml);
242                                    infoWindow.open(currentMap, marker.getPosition());
243                                });
244
245                                return marker;
246                                });
247
248                                if (orderMarkers.length > 0) {
249                                    currentMap.add(orderMarkers);
250                                }
251
252                                setLoading(false);

```


5.5 志愿服务执行与积分



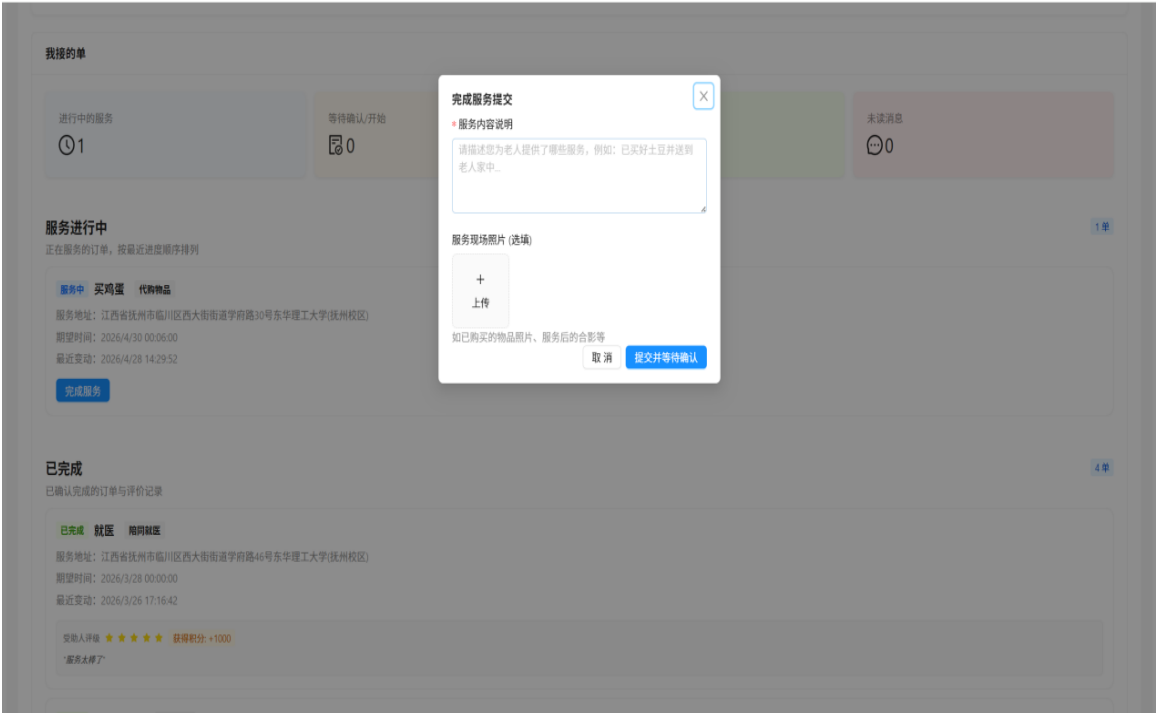


图 5-6 志愿服务执行与积分

如图 5-6 所示，志愿服务执行与积分模块用于支撑志愿者从接单后的服务开展到服务结束后的结果反馈这一完整过程。志愿者在执行服务时可以查看工单详情、沟通记录和服务对象信息，完成服务后再上传相关凭证、填写服务说明，并由系统同步更新订单状态。与此同时，平台按照既定积分规则对有效服务行为进行记录和结算，将服务时长、完成质量与激励机制联系起来。该设计既保证了服务过程可追溯，也有助于提高志愿者参与积极性，形成较为完整的服务闭环。

其中实现服务状态更新与积分结算逻辑的关键代码如下：

代码清单 5-5 志愿服务执行与积分关键代码

```
// server/src/controllers/orderController.ts

179         return res.status(404).json({ message: 'Order not found' });
180     }
181
182     if (order.status !== 'pending') {
183         return res.status(400).json({ message: 'Order is no longer available' });
184     }
185
186     order.status = 'accepted';
187     order.volunteerId = user.id;
188     await order.save();
189
```

```
190      // Notify the specific elderly user their order was accepted
191      getIO().to(`user_${order.elderlyId}`).emit('order_status_update', order);
192
193      res.json({ message: 'Order accepted successfully', order });
194
195    } catch (error) {
196      console.error('Accept order error:', error);
197      res.status(500).json({ message: 'Server error' });
198    }
199  };
200
201  export const updateOrderStatus = async (req: AuthRequest, res: Response) => {
202    try {
203      const user = req.user;
204      const { id } = req.params;
205      const { status, completionPhotos, completionDescription } = req.body; // 'in_progress',
'submitted', 'completed'
206
207      const order = await Order.findByPk(parseInt(id as string, 10));
208
209      if (!order) {
210        return res.status(404).json({ message: 'Order not found' });
211      }
212
213      // Only the assigned volunteer or admin can update status
214      if (user.role === 'volunteer') {
215        if (order.volunteerId !== user.id) {
216          return res.status(403).json({ message: 'Not authorized for this order' });
217        }
218        if (status === 'completed') {
219          return res.status(403).json({ message: '志愿者不能直接完成订单，请提交并等待确认'
' });
220        }
221      }
222
223      order.status = status;
224      if (completionPhotos) {
225        order.completionPhotos = JSON.stringify(completionPhotos);
226      }
227      if (completionDescription) {
228        order.completionDescription = completionDescription;
229      }
230      if (status === 'submitted') {
```

```
// server/src/services/pointService.ts

10   'default': 1
11 };
12
13 export const awardPointsForOrder = async (volunteerId: number, orderId: number, category:
string, pointsReward?: number | null) => {
14   const pointsToAward = pointsReward !== null ? pointsReward : (POINT_RULES[category] ||
POINT_RULES['default']);
15
16   const transaction = await sequelize.transaction();
17   try {
18     // 1. Add points to User
19     const user = await User.findByPk(volunteerId, { transaction });
20     if (!user) throw new Error('Volunteer not found');
21
22     user.points += pointsToAward;
23     await user.save({ transaction });
24
25     // 2. Create PointLog
26     await PointLog.create({
27       userId: volunteerId,
28       pointsChanged: pointsToAward,
29       reason: `Completed order: ${category}`,
30       orderId: orderId
31     }, { transaction });
32
33     await transaction.commit();
34     console.log(`Successfully awarded ${pointsToAward} points to volunteer
${volunteerId}`);
35   } catch (error) {
36     await transaction.rollback();
37     console.error('Failed to award points transaction:', error);
38     throw error;

```

5.6 管理员数据看板



图 5-7 管理员数据看板

如图 5-7 所示，管理员数据看板主要用于集中展示平台运行过程中的关键指标，包括用户规模、订单数量、服务完成情况、活跃趋势以及其他统计结果等内容。系统通过对业务数据进行汇总、分类和可视化处理，将原本分散在不同模块中的信息以图表和数字概览的形式统一呈现，便于管理员快速把握平台整体运行状态。从技术实现角度看，该模块通常依赖后端聚合查询与前端图表组件协同完成数据展示，不仅能够提高信息读取效率，也为后续运营分析、资源调度和管理决策提供了较强的数据支撑。

其中实现管理员数据看板聚合统计接口的关键代码如下：

代码清单 5-6 管理员数据看板关键代码

```
// server/src/controllers/adminController.ts
9
10 export const getDashboardStats = async (req: AuthRequest, res: Response) => {
11   try {
12     if (req.user.role !== 'admin') {
13       return res.status(403).json({ message: 'Admin access required' });
14     }
15
16     const todayStart = dayjs().startOf('day').toDate();
17     const monthStart = dayjs().startOf('month').toDate();
18
19     // Pending orders
20     const pendingOrdersCount = await Order.count({ where: { status: 'pending' } });
```

```
21
22      // Month Completion rate: (Orders created this month AND completed) / (Orders created
this month)
23      const totalMonthOrders = await Order.count({ where: { createdAt: { [Op.gte]:
monthStart } } });
24      const completedMonthOrders = await Order.count({
25        where: {
26          createdAt: { [Op.gte]: monthStart },
27          status: 'completed'
28        }
29      });
30
31      const completionRate = totalMonthOrders === 0 ? 0 : (completedMonthOrders /
totalMonthOrders) * 100;
32
33      // Categories distribution (for ECharts Pie chart)
34      const categoriesData = await Order.findAll({
35        attributes: ['category', [sequelize.fn('COUNT', sequelize.col('id')), 'count']],
36        group: ['category']
37      });
38
39      // User stats
40      const totalUsers = await User.count();
41      const newUsersToday = await User.count({ where: { createdAt: { [Op.gte]:
todayStart } } });
42
43      res.json({
44        stats: {
45          pendingOrdersCount,
46          completionRate: parseFloat(completionRate.toFixed(2)),
47          categoriesData,
48          totalUsers,
49          newUsersToday
```

5.7 志愿者审核管理

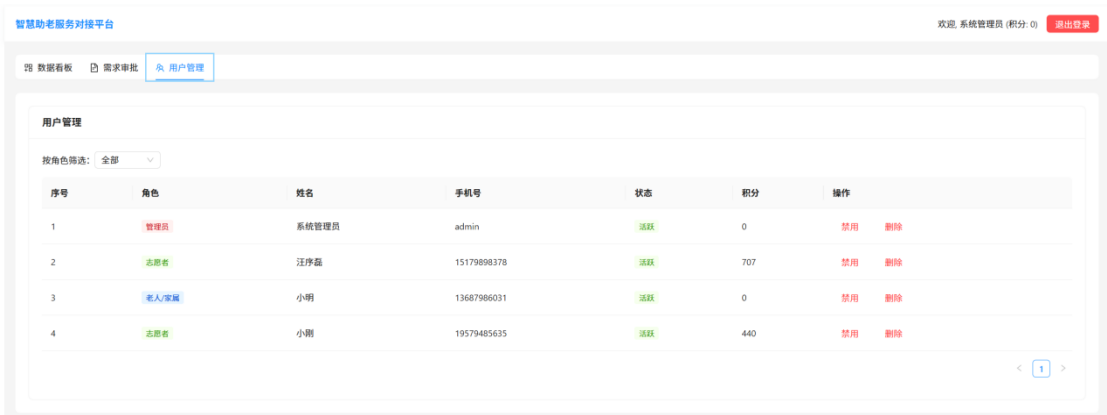


图 5-8 志愿者审核管理

如图 5-8 所示，志愿者审核管理模块主要面向管理员开放，用于处理新注册志愿者的资格审查工作。管理员可以在该页面查看申请人的基本信息、认证资料以及相关提交内容，并根据审核规则执行通过、驳回或补充材料等操作。后端在这一过程中负责维护审核状态、记录处理结果并同步更新志愿者账号的可用权限，从而保证只有符合条件的人员才能进入平台参与服务。该模块的设置有助于提升志愿者队伍的真实性与可靠性，为平台后续服务质量控制和安全管理奠定基础。

其中实现志愿者审核与账号状态维护的关键代码如下：

代码清单 5-7 志愿者审核管理关键代码

```
// client/src/pages/admin/UserManagement.tsx

27         setLoading(false);
28     }
29 }, [roleFilter]);
30
31 useEffect(() => {
32     fetchUsers();
33 }, [fetchUsers]);
34
35 const handleStatusUpdate = async (id: number, status: string) => {
36     try {
37
38         // ...
39
40         // ...
41
42         // ...
43
44         // ...
45
46         // ...
47
48         // ...
49
50         // ...
51
52         // ...
53
54         // ...
55
56         // ...
57
58         // ...
59
60         // ...
61
62         // ...
63
64         // ...
65
66         // ...
67
68         // ...
69
70         // ...
71
72         // ...
73
74         // ...
75
76         // ...
77
78         // ...
79
80         // ...
81
82         const colorMap: Record<string, string> = { pending: 'orange', active: 'green',
disabled: 'red' };
83         return <Tag color={colorMap[status]}>{statusMap[status]}</Tag>;
84     },
85     { title: '积分', dataIndex: 'points', key: 'points' },
86     { title: '操作', key: 'action', render: (_, unknown, record: User) => (
```

```
86         <Space size="middle">
87             {record.status === 'pending' && <Button type="link" onClick={() =>
handleStatusUpdate(record.id, 'active')}>审核通过</Button>}
88             {record.status !== 'disabled' && <Button type="link" danger onClick={() =>
handleStatusUpdate(record.id, 'disabled')}>禁用</Button>}
89             {record.status === 'disabled' && <Button type="link" onClick={() =>
handleStatusUpdate(record.id, 'active')}>解禁</Button>}
90             <Button type="link" danger onClick={() => handleDelete(record)}>删除</Button>
91         </Space>

// server/src/controllers/adminController.ts

81 export const updateUserStatus = async (req: AuthRequest, res: Response) => {
82     try {
83         if (req.user.role !== 'admin') {
84             return res.status(403).json({ message: 'Admin access required' });
85         }
86
87         const { id } = req.params;
88         const { status } = req.body; // 'active', 'disabled'
89
90         const user = await User.findByPk(parseInt(id as string, 10));
91         if (!user) {
92             return res.status(404).json({ message: 'User not found' });
93         }
94
95         user.status = status;
96         await user.save();
97
98         res.json({ message: 'User status updated successfully', user });
99     } catch (error) {
100         console.error('Update user status Error:', error);
101         res.status(500).json({ message: 'Server error' });
102     }
103 };
104
105 export const deleteUser = async (req: AuthRequest, res: Response) => {
```


5.8 需求审核与工单调度

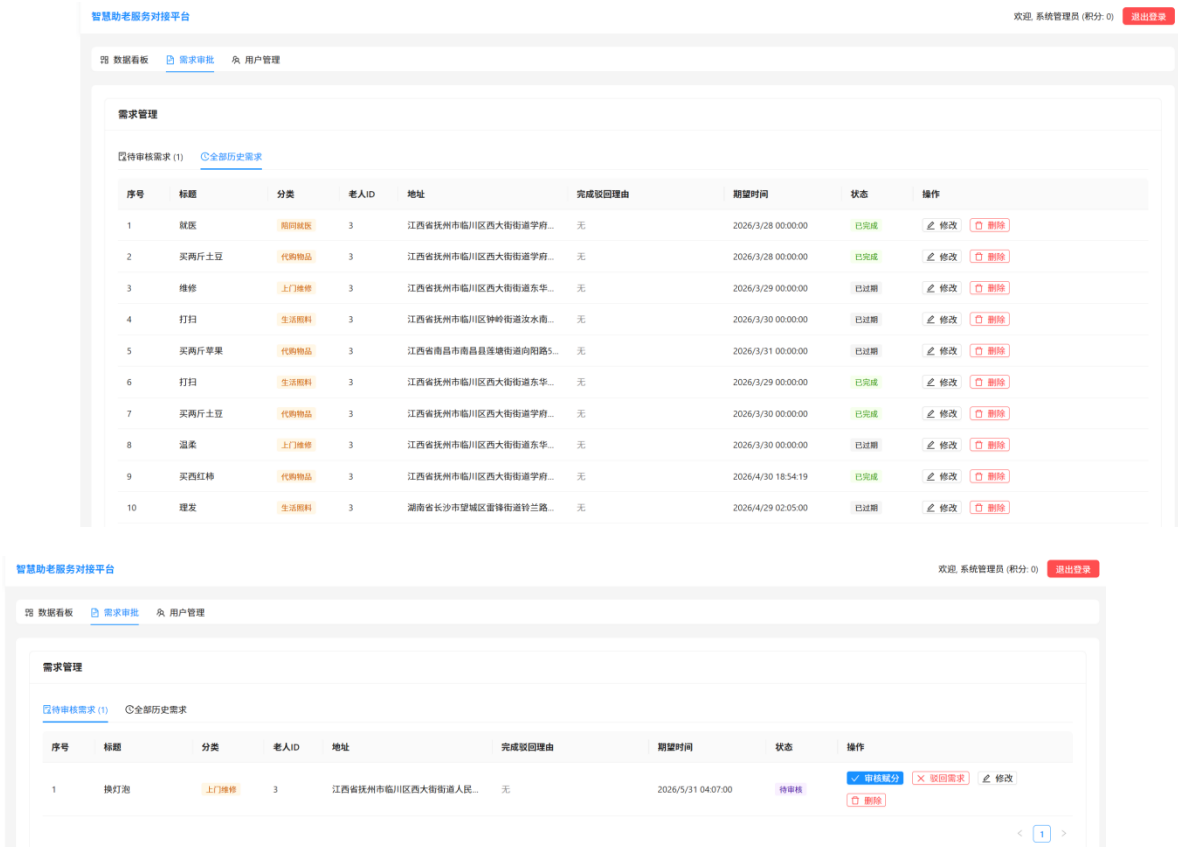


图 5-9 需求审核与工单调度

如图 5-9 所示，需求审核与工单调度模块是平台后台管理流程中的关键环节。管理员在收到用户提交的服务需求后，需要对需求内容、时间安排、地址信息以及服务可行性进行审核，并根据实际情况选择合适的志愿者或服务资源进行工单分配。系统在这一过程中不仅要维护订单状态流转，还要保证审核结果、调度信息与前台显示内容保持一致，从而避免信息不同步问题。通过该模块，平台能够实现需求受理、任务派发和执行衔接的规范化管理，提高整体业务处理效率。

其中实现需求审核、赋分与工单调度的关键代码如下：

代码清单 5-8 需求审核与工单调度关键代码

```
// client/src/pages/admin/OrderAudit.tsx

89      }
90
91      if (!pointsReward || pointsReward <= 0) {
92          message.warning('请输入有效的奖励积分');
93          return;
94      }
```

```
95
96     setActionLoading(true);
97     try {
98         await api.put(`/admin/orders/${selectedOrder.id}/audit`, {
99             status: 'pending',
100             pointsReward,
101         });
102         message.success('审核通过，已放入任务大厅');
103         setAuditModalOpen(false);
104         fetchOrders();
105     } catch (error) {
106         const err = error as { response?: { data?: { message?: string } } };
107         message.error(err.response?.data?.message || '审核失败');
108     } finally {
109         setActionLoading(false);
110     }
111 };
112
113 const handleDisputeAction = (order: Order, nextStatus: 'pending' | 'submitted') => {
114     const isReturnToHall = nextStatus === 'pending';
115     Modal.confirm({
116         title: isReturnToHall ? '放回任务大厅' : '继续交由家属确认',
117
118         // server/src/controllers/adminController.ts
119
120     });
121 }
122
123 await user.destroy();
124 res.json({ message: 'User deleted successfully' });
125 } catch (error) {
126     console.error('Delete user Error:', error);
127     res.status(500).json({ message: 'Server error' });
128 }
129 };
130
131 export const auditOrder = async (req: AuthRequest, res: Response) => {
132     try {
133         if (req.user.role !== 'admin') {
134             return res.status(403).json({ message: 'Admin access required' });
135         }
136
137         const { id } = req.params;
138         const { status, pointsReward } = req.body; // 'pending', 'rejected', 'submitted'
```

```
139
140     if (status !== 'pending' && status !== 'rejected' && status !== 'submitted') {
141         return res.status(400).json({ message: 'Invalid audit status' });
142     }
143
144     const order = await Order.findByPk(parseInt(id as string, 10));
145
146     if (!order) {
147         return res.status(404).json({ message: 'Order not found' });
148     }
149
150     if (order.status !== 'under_review') {
151         return res.status(400).json({ message: 'Order is not under review' });
152     }
153
154     const isCompletionDispute = Boolean(order.volunteerId &&
order.completionRejectionReason);
155
156     if (!isCompletionDispute && status === 'submitted') {
157         return res.status(400).json({ message: 'Only disputed completion orders can be
returned for family reconfirmation' });
158     }
159
160     if (!isCompletionDispute && status === 'pending' && (pointsReward === undefined ||
pointsReward === null || pointsReward <= 0)) {
161         return res.status(400).json({ message: 'Approved orders must have a valid points
reward' });
162     }
163
164     if (isCompletionDispute) {
165         if (status === 'pending') {
166             order.status = 'pending';
167             order.volunteerId = null;
168             order.completionDescription = null;
169             order.completionPhotos = null;
170             order.completionRejectionReason = null;
171         } else if (status === 'submitted') {
172             order.status = 'submitted';
173             order.completionRejectionReason = null;
174         } else {
175             order.status = 'rejected';
176         }
177     } else {
```

```
178         order.status = status;
179         if (status === 'pending') {
180             order.pointsReward = pointsReward;
181         }
182     }
183 }
```

5.9 统计分析



图 5-10 统计分析

如图 5-10 所示，统计分析页面主要围绕平台运行过程中形成的订单、服务、积分和用户行为等数据开展汇总与展示。系统可按照时间区间、业务类别或其他筛选条件对数据进行分类对比，使管理人员能够更直观地观察平台在不同阶段的运行变化情况。借助图表化展示方式，平台可以较快发现高频服务类型、活跃时段以及资源配置中的潜在问题，为后续服务优化、人员调度和管理策略调整提供依据。该模块体现了系统从基础业务处理向数据辅助决策延伸的实现思路。

其中实现统计分析图表渲染的关键代码如下：

代码清单 5-9 统计分析关键代码

```
// client/src/pages/admin/Dashboard.tsx
19
20 const Dashboard: React.FC = () => {
21     const [stats, setStats] = useState<DashboardStats | null>(null);
22
23     useEffect(() => {
24         const fetchStats = async () => {
25             try {
```

```
26         const { data } = await api.get('/admin/dashboard');
27         setStats(data.stats);
28     } catch {
29         message.error('Failed to load dashboard stats');
30     }
31 };
32 fetchStats();
33 }, []);
34
35 const getOptions = () => {
36     if (!stats) return {};
37     const data = stats.categoriesData.map((d: CategoryData) => ({
38         name: d.category,
39         value: d.count
40     }));
41
42     return {
43         title: { text: '需求类型分布', left: 'center' },
44         tooltip: { trigger: 'item' },
45         legend: { orient: 'vertical', left: 'left' },
46         series: [
47             {
48                 <Statistic
49                     title="本月订单完成率"
50                     value={stats?.completionRate || 0}
51                     suffix="%"
52                     prefix={<CheckCircleOutlined />}
53                     valueStyle={{ color: '#52c41a' }}
54                 />
55             </Card>
56         </Col>
57         <Col xs={24} sm={8}>
```

5.10 个人信息与老人档案

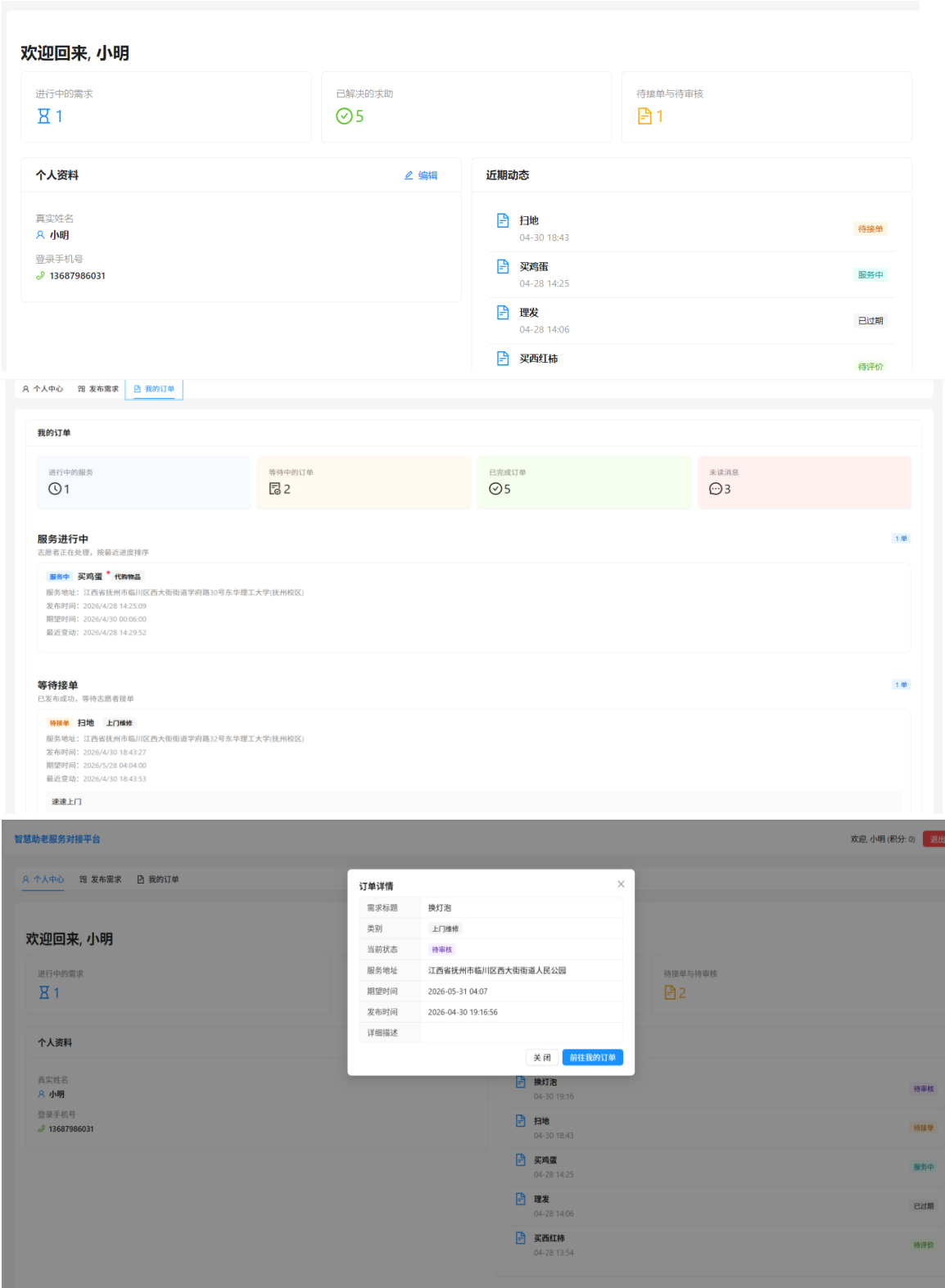


图 5-11 个人信息与老人档案

如图 5-11 所示,个人信息与老人档案模块用于维护平台用户的基础资料以及老人群体的照护相关信息,是实现个性化服务匹配的重要数据基础。用户可以在

该页面中补充或修改联系方式、身份信息、健康状况、特殊需求等内容，系统则负责对相关信息进行存储、更新和调用。当前端在需求发布、服务分配和沟通过程中需要展示特定对象信息时，均可通过该档案模块提供的数据进行支撑。该模块不仅提高了信息管理的完整性，也有助于平台在后续服务过程中提供更加准确和有针对性的帮助。

其中实现个人信息与老人档案读取、更新的关键代码如下：

代码清单 5-10 个人信息与老人档案关键代码

```
// server/src/controllers/userController.ts

6  export const getProfile = async (req: AuthRequest, res: Response) => {
7    try {
8      const user = await User.findByPk(req.user.id, {
9        attributes: ['id', 'name', 'phone', 'address', 'role', 'points'],
10     });
11
12     if (!user) {
13       return res.status(404).json({ message: 'User not found' });
14     }
15
16     // Find redemptions where the current user's phone is the target phone
17     const redemptions = await PrizeRedemption.findAll({
18       where: { targetPhone: user.phone },
19       order: [['createdAt', 'DESC']]
20     });
21
22     const userData = user.get({ plain: true });
23     userData.prizeRedemptions = redemptions;
24
25     res.json({ user: userData });
26   } catch (error) {
27     console.error('getProfile error:', error);
28     res.status(500).json({ message: 'Server error fetching profile' });
29
30   }
31
32   export const updateProfile = async (req: AuthRequest, res: Response) => {
33     try {
34       const { name, phone, address } = req.body;
35       const user = await User.findByPk(req.user.id);
36
37       if (!user) {
38         return res.status(404).json({ message: 'User not found' });
39       }
40
41       // If updating phone, ensure it's not taken by someone else
42       if (phone && phone !== user.phone) {
43         const existing = await User.findOne({ where: { phone } });
44         if (existing) {
```



```
45         return res.status(400).json({ message: '此手机号已被其他账号使用' });
46     }
47 }
48
49     user.name = name !== undefined ? name : user.name;
50     user.phone = phone !== undefined ? phone : user.phone;
51     user.address = address !== undefined ? address : user.address;
52
53     await user.save();
54
55     res.json({ message: 'Profile updated successfully', user });
56 } catch (error) {
57     console.error('updateProfile error:', error);
58     res.status(500).json({ message: 'Server error updating profile' });
59 }

```

// client/src/pages/volunteer/Dashboard.tsx

```
122     try {
123         const { data: mine } = await api.get('/orders', { params: { tab: 'mine' } });
124         setMyOrders(mine.orders);
125
126         const { data: profileRes } = await api.get('/auth/profile');
127         setProfileDataObj(profileRes.user);
128         form.setFieldsValue({
129             name: profileRes.user.name,
130             phone: profileRes.user.phone,
131             address: profileRes.user.address,
132         });
133
134         if (profileRes.user.points !== user?.points) {
135             updateUser({ points: profileRes.user.points });
136         }
137     } catch {
138         message.error('数据加载失败');
139     } finally {

```

第 6 章 系统测试

6.1 测试目的

系统测试的目的不单是检验社区助老服务对接平台各个模块的功能能否正常运行，更重要的是从完整的业务链条出发来检验平台在真实的使用环境中稳定性、协同性、适用性。由于本系统包含老年人或者家属、志愿者和管理员这三个角色，并且业务流程涵盖需求发布、审核调度、接单执行、聊天沟通、积分结算和评价反馈等各个方面，所以测试工作要重点放在各个角色之间数据传递是否正确、状态流转是否清楚、界面反馈是否及时上。经过系统的测试之后，可以找出平台在权限控制、异常提示、表单校验、接口交互、实时通信等各方面存在的问题，从而给后续的优化完善提供依据。

6.2 测试方法

为了使系统测试内容更直观，本文将测试方法、适用范围以及测试覆盖情况采用表格方式进行展示。各项测试围绕老年人或者家属端、志愿者端和管理员端的典型业务展开，重点关注功能可用性、流程完整性以及异常场景下的系统反馈。如表 6-1 所示。

表 6-1 测试方法与适用范围

测试类型	主要目标	适用范围
功能测试	验证登录、需求发布、接单、审核、聊天、积分等功能是否可正常使用	老年人或者家属端、志愿者端、管理员端
流程联动测试	验证需求从发布、审核、接单到完成评价的完整业务链条是否顺畅	订单流转、状态同步、角色协同
异常场景测试	验证空值提交、越权访问、重复操作、消息延迟等情况下的系统反馈	表单校验、权限控制、接口返回、实时通信

测试方法与适用范围表概括了本章采用的主要测试思路，分别从功能测试、流程联动测试和异常场景测试三个层面展开，覆盖了系统核心业务功能、跨角色协同流程以及异常输入和通信延迟等边界情况，为后续测试用例设计提供了方法依据。如表 6-2 所示。

表 6-2 测试覆盖内容统计

测试对象	核心模块	验证重点	预期效果
老年人或者家属端	注册登录、老人档案、需求发布、订单跟踪	信息填写完整、状态展示清晰、操作提示明确	能够顺利完成需求提交并查看服务进度
志愿者端	任务大厅、任务地图、接单处理、服务完成、积分奖励	任务筛选准确、状态更新及时、积分累计正确	能够顺利接单并完成服务闭环
管理员端	用户审核、需求审核、工单调度、数据统计	权限控制有效、审核流程规范、统计结果真实	能够稳定完成后台管理与调度工作
异常流程	空值提交、越权访问、重复操作、消息同步	错误提示准确、系统状态一致、异常恢复合理	系统具备基本容错与反馈能力

测试覆盖内容统计表从测试对象角度梳理了系统在不同角色和异常流程下的验证范围，其中老年人或者家属端重点关注注册登录、档案维护、需求发布与订单跟踪，志愿者端重点关注接单处理、服务执行和积分奖励，管理员端重点关注审核调度与数据统计；同时还补充了空值提交、越权访问、重复操作和消息同步等异常流程测试，用于说明系统在典型业务场景和边界场景下均具备较为完整的功能覆盖。

6.3 测试用例

测试覆盖内容统计表从测试对象角度说明了不同端口和异常流程的验证范围，其中老年人或者家属端、志愿者端、管理员端分别对应各自核心业务模块，预期效果则用于衡量系统在典型使用场景下是否具备完整、稳定的业务支撑能力。

表 6-3 注册登录与权限控制模块测试用例

用例编号	测试项目	前置条件/输入	操作步骤	预期结果	实际结果
L1	用户注册	输入手机号、密码与角色信息	提交注册表单	系统完成注册并提示成功	符合预期
	用户登录	账号已完成注册	输入账号密码后登录	系统跳转到对应角色首页	符合预期
L2					
L3	身份区分跳转	存在家属端、志愿者端、管理员端账号	分别使用三类账号登录	系统按角色进入对应功能界面	符合预期
L4	异常密码校验	输入错误密码	提交登录请求	系统提示账号或密码错误	符合预期
L5	未登录访问拦截	浏览器直接访问受限页面	输入受限地址访问页面	系统拦截并返回登录页	符合预期

注册登录与权限控制模块测试用例表展示了系统在用户认证与访问控制方面的关键验证结果，涵盖注册、登录、角色跳转、密码校验和未登录拦截等场景；各用例实际结果均符合预期，说明系统具备基本的身份识别与权限隔离能力。如表 6-3 所示。

表 6-4 需求发布与订单跟踪模块测试用例

用例编号	测试项目	前置条件/输入	操作步骤	预期结果	实际结果
D1	需求发布	家属端用户已登录，填写老人信息与需求内容	提交需求表单	系统生成新订单并进入待审核状态	符合预期
D2	订单跟踪	已存在待审核或进行中的订单	进入订单跟踪页面查看详情	系统正确显示当前状态与进度	符合预期

续表 6-4

用例编号	测试项目	前置条件/输入	操作步骤	预期结果	实际结果
D3	需求内容校验	缺少联系人或服务时间等必填项	提交需求信息	系统给出必填项提示并阻止提交	符合预期
D4	修改需求	订单处于可编辑状态	进入订单详情后修改需求内容	系统保存修改结果并同步状态	符合预期
D5	历史订单查询	用户账户下存在多条订单记录	切换至历史订单列表	系统按时间展示历史服务记录	符合预期

需求发布与订单跟踪模块测试用例表重点验证家属端从提交服务需求到查看历史订单的主要业务流程，其中对必填项校验、订单修改和进度跟踪等环节均进行了覆盖，能够反映需求管理功能在真实使用场景中的完整性与可用性。如表 6-4 所示

表 6-5 志愿者接单与服务执行模块测试用例

用例编号	测试项目	前置条件/输入	操作步骤	预期结果	实际结果
V1	任务大厅接单	志愿者账号已登录且存在待接订单	在任务大厅选择任务并点击接单	系统接单成功并更新订单状态	符合预期
V2	任务地图查看	存在带服务地址的待接任务	切换至地图视图查看任务分布	系统正确显示任务点位与基础信息	符合预期
V3	服务完成提交	志愿者已接单并完成上门服务	填写服务结果并提交完成	系统更新订单为待确认或已完成	符合预期
V4	积分奖励更新	订单已完成且满足积分规则	刷新个人中心或积分页面	系统正确累计志愿服务积分	符合预期
V5	重复接单限制	某任务已被其他志愿者接取	再次尝试点击接单	系统提示任务已被接单不可重复	符合预期

志愿者接单与服务执行模块测试用例表主要反映志愿者端在任务获取、地图

查看、服务完成以及积分累计等环节的运行情况，同时通过重复接单限制验证了系统对并发业务冲突的控制效果，说明该模块能够支撑较为完整的服务闭环。如表 6-5 所示。

表 6-6 后台审核调度与实时通信模块测试用例

用例编号	测试项目	前置条件/输入	操作步骤	预期结果	实际结果
M1	后台需求审核	管理员已登录且存在待审核需求	进入审核页面执行通过或驳回	系统正确记录审核结果并更新状态	符合预期
M2	工单调度	存在待分配或需协调的订单	管理员调整订单信息并分配任务	系统完成调度并同步到相关角色	符合预期
M3	实时聊天	订单双方均在线并进入聊天界面	发送文本消息进行沟通	消息实时送达并正确显示会话内容	符合预期
M4	离线消息恢复	一方短暂离线后重新进入聊天页	重新打开对应会话	系统加载历史消息并恢复沟通记录	符合预期
M5	越权操作拦截	普通用户尝试访问后台管理地址	直接输入后台页面路径	系统拒绝访问并提示无权限	符合预期

后台审核调度与实时通信模块测试用例表用于验证管理员审核、工单调度以及订单相关聊天功能的稳定性，并补充了离线消息恢复和越权访问拦截等场景；从测试结果看，系统在后台管理协同和通信交互方面能够保持较好的正确性与一致性。如表 6-6 所示。

6.4 测试结论

经过多次的功能测试以及场景联动测试可知，社区助老服务对接平台的总体实现情况基本上达到了预期的目的。测试结果表明，系统可以较好地支持三类角色在各自的权限范围内完成相应的操作，登录认证、需求发布、订单审核、任务接单、服务执行、积分奖励和聊天沟通等主要功能都能正常运行，订单状态在各个阶段之间的转换也比较清楚。而且前后端分离架构、实时通信机制在实际测试中也具有较好的可用性，说明用 Node.js、React、MySQL、Socket.io 这些技术方案来完成本课题的开发是可行的。测试中发现系统对于老年用户提示有不足，对于异常情况说明不够清楚，界面细节不统一等，可以继续从交互友好、性能稳定、

业务智能三个方向进行改善。总体来说,本系统已经具备了比较完整的应用基础,可以为社区助老服务数字化管理提供可以参照的实现方案。

结 论

本文就社区养老服务中存在“需求难连接、过程难跟踪、资源难协同”等现实问题，根据目前社区养老服务数字化转型的需求来设计并实现了一个基于 Node.js、React 的社区助老服务对接平台。本文在课题研究背景与意义的基础上，从国内外的研究现状入手，对系统的功能进行了详细的分析，设计出系统的总体结构，确定了系统的各个模块，并且对数据库进行了设计，最后完成了系统的实现与测试验证。本文主要对需求发布、服务匹配、工单审核、任务执行、过程沟通、积分激励、平台管理等环节进行完整的业务闭环。

从系统实现的结果看，该平台较好地支持了老年人或者家属端、志愿者端和管理员端三个角色的协同操作，在登录认证、老人档案管理、需求发布、订单跟踪、志愿者接单、实时聊天、审核调度、积分奖励和数据统计等各个方面都达到了预期的效果。测试结果表明，在主要的业务流程中，系统可以保持比较清晰的状态流转和数据传递逻辑，说明使用前后端分离架构、Node.js 服务端、React 前端框架、MySQL 数据存储和 Socket.io 实时通信机制的技术路线，可以较好的满足社区助老服务平台在交互响应、业务协同和后续扩展方面的实际需要。

本文完成的系统实现证明了社区助老服务对接平台在提高服务供需匹配效率、增强过程透明度、优化社区管理协同能力等方面有较好的可行性以及应用价值。但是由于时间、数据来源、应用场景范围等因素的限制，目前系统在智能推荐、适老化细节打磨、多源服务资源接入、异常预警、高并发场景性能优化等各方面还有待进一步提高。之后可以继续对精准服务推荐、更加友好的适老化交互设计、社区养老资源深度整合等方向进行扩展研究，从而提高平台的实用性、稳定性和推广价值。

致 谢

本次毕业设计和论文撰写过程中,得到很多老师的指导和支持,感谢他们。课题从选题确定、需求分析、系统设计、前后端功能实现、论文结构梳理和内容修改等各个环节都离不开别人的帮助和支持。本次毕业设计使我对 Node.js、React、数据库的设计以及前后端的协同开发有更深入的认识,并且更加体会到了独立思考、不断修改、规范表达对于毕业设计的重要性。

首先要向指导老师在毕业设计过程中给予的悉心指导表示深深的感谢。不论是课题方向选择、系统功能取舍、论文框架搭建、具体实现细节和写作规范等各方面,老师都给了我很多中肯的、具体的建议,使我在不断地发现问题、改进自己的不足之处、从而完善作品的过程中有所提高。同时也要感谢学院有关课程教师在大学期间对我的专业基础能力进行培养,为本次课题的顺利完成打下了良好的基础。

再次感谢为本项目开发以及撰写论文过程中,给予过我帮助的同学、朋友。大家在资料查找、功能测试、界面体验反馈、论文修改建议等各方面都给我提供了帮助,在我遇到困难的时候及时调整思路继续前进。

最后要感谢家人一直以来的理解、支持和鼓励,使我在比较稳定的情况下完成了本次毕业设计和论文的撰写。在此谨向关心、帮助过我的老师、同学、朋友以及家人表示衷心的感谢。

参考文献

- [1] 朱培垚, 乔雯, 张佳萌. 基于区块链技术的中国智慧养老产业发展研究[J]. 对外经贸, 2025, (11):110-114. DOI:10.20216/j.cnki.fert1987.2025.11.023.
- [2] 莫文卓, 刘春艳, 胡骏杰, 等. 基于用户体验调查的“乐龄伴行”智慧助老 APP 开发与设计[J]. 内江科技, 2025, 46(03):80-82.
- [3] 凌超, 方卫青, 齐梦婕. 智慧助老平台的设计与实现[J]. 电脑知识与技术, 2025, 21(05):110-115+118. DOI:10.14004/j.cnki.ckt.2025.0213.
- [4] 沈进兵. 社区教育何以助老: 基于社会行动理论的分析[J]. 教育与职业, 2023, (23):54-62. DOI:10.13615/j.cnki.1004-3985.2023.23.001.
- [5] 彭敏学, 程鲲, 张海旭. 基于微信小程序的社区智慧助老信息服务系统设计[J]. 工业设计, 2023, (09):105-108.
- [6] 钱荣华. 基于“互联网+”的智慧助老空中课堂建设研究[J]. 中国成人教育, 2023, (05):47-50.
- [7] 傅俊哲, 李俊杰. 基于 Vue 的助老微信小程序的设计与实现[J]. 电脑知识与技术, 2023, 19(07):58-60. DOI:10.14004/j.cnki.ckt.2023.0443.
- [8] 耿梦瑶, 杨宁. 设计心理学在智慧助老产品设计中的应用研究[J]. 工业设计, 2022, (10):55-57.
- [9] 赵敏. 智慧助老视角下提升老年人智能手机使用能力的实务研究[D]. 青海师范大学, 2022. DOI:10.27778/d.cnki.gqhzy.2022.000702.
- [10] 王皖琳, 梁蓝芋, 黄红梅, 等. 智慧医疗背景下老年群体“数字鸿沟”现状分析及适老化改造对策研究[J]. 华西医学, 2022, 37(04):586-591.
- [11] 赵莹德. 基于物联网的智慧社区服务平台实现[J]. 新型工业化, 2021, 11(03):71-74.
- [12] 张致瑜, 王霖, 黄立平, 等. 智慧平台及智慧社区服务体系构建[J]. 中国新技术新产品, 2021, (24):142-145.
- [13] 樊斐. 智慧健康养老服务平台基于微服务架构的实现[J]. 信息技术与标准化, 2020, (11):67-70+75.
- [14] 王晨. 基于智慧社区的手机律师服务平台设计与实现[D]. 中南民族大学, 2022.
- [15] 靳冰, 平雷. 基于 AI+IoT 的智慧服务区综合管理平台[J]. 中国交通信息化, 2021, (06):39-41.
- [16] 张凯. 基于智慧社区的居家养老服务模式探索[J]. 现代营销(经营版), 2020, (03):46-47.
- [17] 温璐亚, 赵袁杰. 基于区块链技术的助老服务平台建设[J]. 经济研究导刊, 2023, (11):116-118.
- [18] 周红婕. 上海社区智慧养老服务管理平台的运营机制研究[D]. 上海工程技术大学, 2021.
- [19] 高天. 智慧社区一体化服务平台的设计与实现[D]. 北京邮电大学, 2023.
- [20] Shi A ,Zhao Y ,Qi Q , et al. Policy Support and Demand Response Strategies for Smart Elderly Care Service Platforms[J].Academic Journal of Humanities & Social Sciences,2026,9(1):DOI:10.25236/AJHSS.2026.090101.
- [21] Tan X ,Su X . Continuity of Nursing Service Platform under "Smart Elderly Care" and Its Application Value in Family Nursing of Elderly Patients after Discharge.[J].Alternative therapies in health and medicine,2024.

- [22] Liu Q ,Jia L ,Yan Q .Design of Intelligent Elderly Care Platform Based on Intelligent Perception Technology[C]//AEIC Academic Exchange Information Center(China).Proceedings of 2nd International Conference on Information Economy, Data Modeling and Cloud Computing(ICIDC 2023).Shandong Institute of Commerce and Technology;;2023:189-195.DOI:10.26914/c.cnkihy.2023.086257.